Ex.No.1: FORMATION OF BUS ADMTTANCE AND BUS IMPEDANCE MATRIX

AIM

To develop a program to obtain Y_{bus} matrix for the given networks by the method of inspection.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. Define sparsity in a matrix.
2. Recall the concept of singular matrix.
3. State the application of cofactor matrix.
4. Differentiate primitive admittance and bus admittance.
5. Define line charging admittance in a power system network

ALGORITHM

Step 1: Initialize [Y-Bus] matrix that is replace all entries by zero.

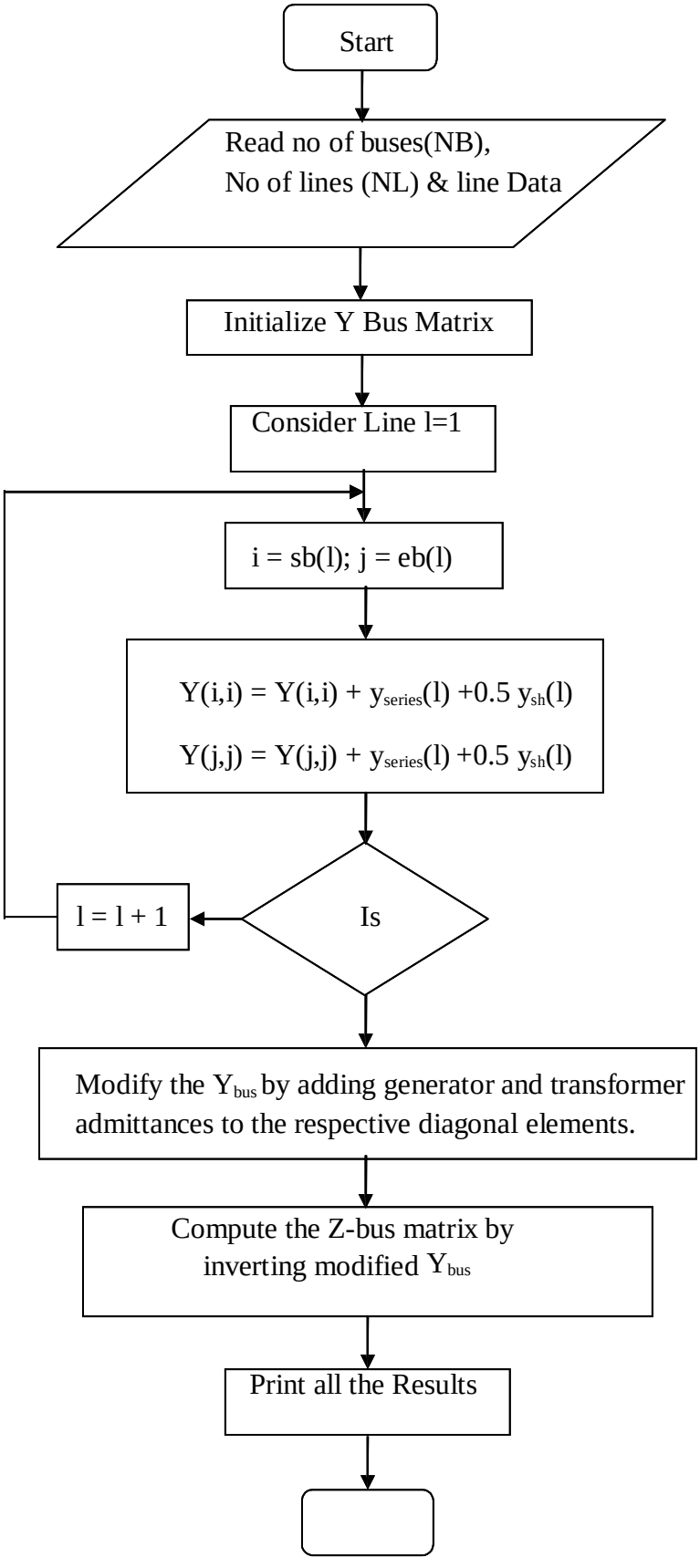
$Y_{ij} = Y_{ij} - y_{ij} = Y_{ji}$ = off diagonal element

Step 2: Compute $Y_{ii} = \sum_{j=1} y_{ij}$ = diagonal element.

Step 3: Modify the Y_{bus} matrix by adding the transformer and the generator admittances to the respective diagonal elements of Y bus matrix

Step 4: Compute the Z-Bus matrix by inverting the modified Y_{bus} matrix

FLOW CHART


Start
Read no of buses(NB),
No of lines (NL) & line Data
Initialize Y Bus Matrix
Consider Line l=1
i = sb(l); j = eb(l)
Y(i,i) = Y(i,i) + y_{series}(l) +0.5 y_{sh}(l)
Y(j,j) = Y(j,j) + y_{series}(l) +0.5 y_{sh}(l)
Is
l = l + 1
Modify the Y_{bus} by adding generator and transformer admittances to the respective diagonal elements.
Compute the Z-bus matrix by inverting modified Y_{bus}
Print all the Results

PROBLEM STATEMENT

The [Y-Bus] matrix is formed by inspection method for a four bus system. The line data and is given below.

LINE DATA

Line Number	SB	EB	Series Impedance (p.u)	Line charging Admittance (p.u)
1	1	2	0.10 + j0.40	j0.015
2	2	3	0.15 + j0.60	j0.020
3	3	4	0.18 + j0.55	j0.018
4	4	1	0.10 + j0.35	j0.012
5	4	2	0.25 + j0.20	j0.030

FORMATION OF Y-BUS BY THE METHOD OF INSPECTION

PROGRAM:

```
clc;
clear all;

% Input number of buses and lines
n = input('Enter number of buses: ');
l = input('Enter number of lines: ');
s = input('Enter 1 for Impedance or 2 for Admittance input: ');

% Initialize matrices
ybus = zeros(n, n);
y = zeros(n, n);       % To store admittance between buses
lc = zeros(n, n);      % Line charging admittance

% Read line data
for i = 1:l
    a = input('Starting bus: ');
    b = input('Ending bus: ');
    t = input('Admittance or Impedance of line: ');
    lca = input('Line charging admittance (total for the line): ');
```

```
    if s == 1
        y(a,b) = 1/t;   % Convert impedance to admittance
    else
        y(a,b) = t;     % Admittance directly given
    end
    y(b,a) = y(a,b);    % Symmetrical for undirected lines

    lc(a,b) = lca;
    lc(b,a) = lca;
end

% Construct Ybus matrix
for i = 1:n
    for j = 1:n
        if i == j
            for k = 1:n
                ybus(i,j) = ybus(i,j) + y(i,k);  % Off-diagonal admittances
                if lc(i,k) ~= 0
                    ybus(i,j) = ybus(i,j) + lc(i,k)/2;  % Add half line charging
                end
            end
        else
            ybus(i,j) = -y(i,j);  % Off-diagonal element is negative admittance
        end
    end
end

% Display Ybus
disp('Ybus matrix is:');
disp(ybus);
```

OUTPUT:

RERSULT:

Ex.No.2: FORMATION OF BUS IMPEDANCE MATRIX

AIM

To develop a program to obtain Y_{bus} matrix for the given networks by the method of inspection.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. Define sparsity in a matrix.
2. Recall the concept of singular matrix.
3. State the application of cofactor matrix.
4. Differentiate primitive admittance and bus admittance.
5. Define line charging admittance in a power system network

ALGORITHM

FORMATION OF Z-BUS BY THE METHOD OF INSPECTION

PROGRAM:

```
clc;
clear all;

n = input('Enter number of buses: ');
l = input('Enter number of lines: ');
s = input('Enter 1 for Impedance or 2 for Admittance: ');

% Initialize matrices
y = zeros(n, n);      % Mutual admittances
lc = zeros(n, n);     % Line charging admittances
ybus = zeros(n, n);   % Bus admittance matrix

% Read line data
for i = 1:l
   a = input('Starting bus: ');
   b = input('Ending bus: ');
   t = input('Admittance or Impedance of line: ');
   lca = input('Line charging admittance (total for the line): ');
```

```
    if s == 1
        y(a,b) = 1/t;  % Convert impedance to admittance
    else
        y(a,b) = t;    % Use admittance directly
    end
    y(b,a) = y(a,b);   % Symmetry

    lc(a,b) = lca;
    lc(b,a) = lca;     % Symmetry
end

% Construct Ybus
for i = 1:n
    for j = 1:n
        if i == j
            for k = 1:n
                ybus(i,j) = ybus(i,j) + y(i,k) + lc(i,k)/2;
            end
        else
            ybus(i,j) = -y(i,j);
        end
    end
end

% Display Ybus
disp('Ybus matrix is:');
disp(ybus);

% Calculate and display Zbus (inverse of Ybus)
zbus = inv(ybus);   % Better numerically than using ybus^(-1)
disp('Zbus matrix is:');
disp(zbus);
```

OUTPUT:

VIVA QUESTIONS:

1. Differentiate primitive admittance and bus admittance in network
2. Differentiate line charging and half line charging admittance based on Ferranti effect
3. List the two applications of Y bus
4. State the applications of Y bus.
5. Indicate the difference between driving point and transfer admittance in Y bus.

STIMULATING QUESTIONS:

1. State the reason for using Z bus in short circuit studies.
2. Why Y bus is sparse and Z bus is not sparse?

RESULT:

Ex.No-3 **FORMATION OF BUS INCIDENCE AND LOOP INCIDENCE MATRIX**
Date:

AIM

To write a suitable program to form a bus incidence and loop incidence matrix for a given power system.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. State the impedance of single line diagram.
2. Differentiate Bus and Node based on network behaviour.
3. Infer the importance of network topology determination.
4. Indicate the order of bus admittance matrix for a given network with ‘n’ nodes.

ALGORITHM

Step 1: Start the program

Step 2: Using the switch cases ask either bus incidence or loop incidence matrix to be found. If loop incidence matrix is to be formed go to step6.

Step 3: Ask number of system.

Step 4: To form bus incidence matrix, rules are
+1= If current is leaving the node
-1= If current is entering the node.
0= if branch is not incident to the node.

Step 5: Using this rule, form bus incidence matrix.

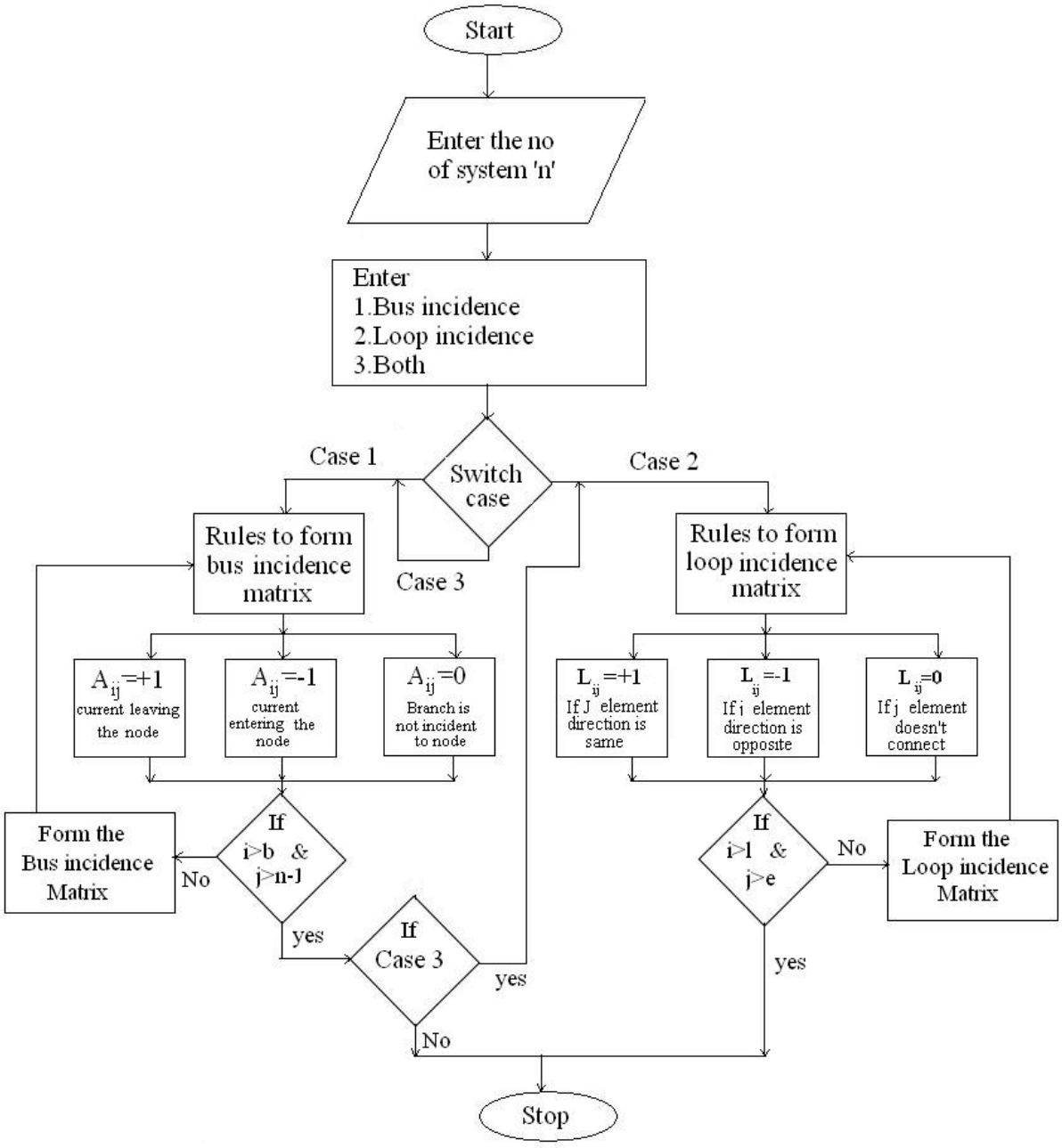
Step 6: To form loop incidence matrix, form loop of a graph drawn, size of the matrix and element node.

Step 7: The direction of the loop is direction of link, the matrix is corresponding element
+1=If the element direction is same as that of its corresponding loop.
-1=If the element is opposite direction.
0=If the element direction does not correspond to that loop.

Step 8: Using the above rules, form the loop incidence matrix and display it.

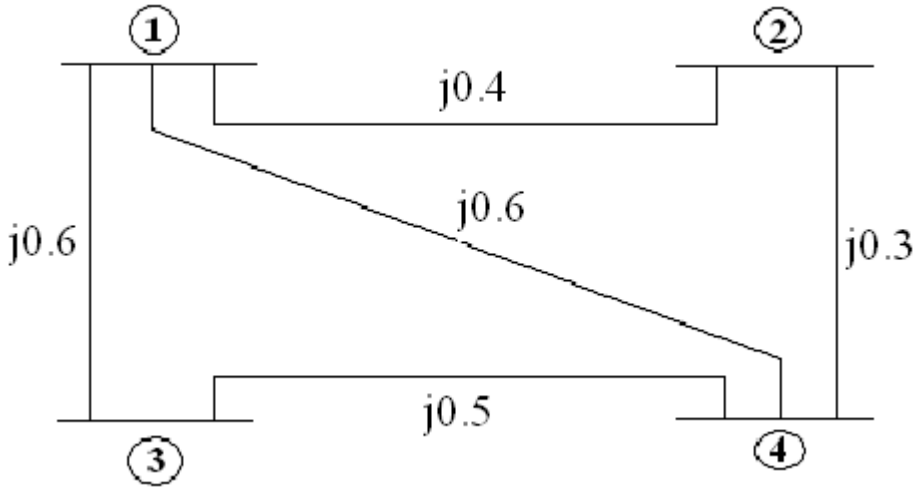
Step 9: Stop the program.

FLOW CHART


Start
Enter the no of system 'n'
Enter
1.Bus incidence
2.Loop incidence
3.Both
Switch case
Case 1
Case 2
Case 3
Rules to form bus incidence matrix
Rules to form loop incidence matrix
Aij=+1 current leaving the node
Aij=-1 current entering the node
Aij=0 Branch is not incident to node
Lij=+1 If J element direction is same
Lij=-1 If i element direction is opposite
Lij=0 If j element doesn't connect
If i>b & j>n-1
No
Form the Bus incidence Matrix
yes
If Case 3
yes
No
If i>1 & j>e
No
Form the Loop incidence Matrix
yes
Stop

PROBLEM STATEMENT

Find the bus incidence matrix [A] and loop incidence matrix [L] for the four bus system shown in figure. Take ground as reference.



BUS INCIDENCE MATRIX PROGRAM:

```
clc;
clear;

n = input('Enter the number of nodes: ');
e = input('Enter the number of elements: ');

a = zeros(e, n);  % Initialize incidence matrix

% Input incidence details using GUI menu
for i = 1:e
   for j = 1:n
      fprintf('\nSelect the incidence of element %d with node %d:\n', i, j);
      reply = menu('Choice', 'Incident Away', 'Incident Towards', 'Not Incident');

      if reply == 1
         a(i,j) = 1;
      elseif reply == 2
         a(i,j) = -1;
      else
         a(i,j) = 0;
      end
   end
end
```

```
ref = input('\nEnter the reference node number: ');

% Display the reduced bus incidence matrix
fprintf('\nThe reduced bus incidence matrix is:\n');
for i = 1:e
   for j = 1:n
      if j ~= ref
         fprintf('%3d\t', a(i,j));
      end
   end
   fprintf('\n');
end
```

LOOP INCIDENCE MATRIX PROGRAM:

```
clc;
clear;

y = input('Enter number of elements: ');
l = input('Enter number of loops: ');

% Initialize loop incidence matrix
c = zeros(y, l);

for i = 1:y
   for j = 1:l
      fprintf('\nIs element %d incident with loop %d?\n', i, j);
      r = input('Enter 1 for YES and 0 for NO: ');

      if r == 1
         fprintf('Is the element %d directed **with** the loop %d?\n', i, j);
         s = input('Enter 1 for YES and 0 for NO: ');

         if s == 1
            c(i, j) = 1;
         else
            c(i, j) = -1;
         end
      else
         c(i, j) = 0;
      end
   end
end

fprintf('\nThe Loop Incidence Matrix is:\n');
disp(c);
```

OUTPUT:

VIVA QUESTIONS:

1. Differentiate Branch and Link in a network
2. Define a tree of the network
3. Indicate the difference between the matrices A and $\hat{A}$
4. Differentiate connected graph and oriented graph of a network
5. State the method for assigning orientation to a network elements

STIMULATING QUESTIONS:

1. Find the number of loops of a network with ‘n’ nodes and ‘m’ elements.
2. State the reason for omitting ground node while forming matrix A.

RESULT:

Ex.No-4 **FORMATION OF BRANCH PATH MATRIX AND CUTSET MATRIX**
Date:

AIM

To write a program to obtain the branch path and cutest matrix using c++ program.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. Indicate the number of branches in a network with 'n' nodes.
2. Infer the significance of primitive admittance of network elements.
3. Differentiate independence and mutually coupled elements.
4. List the merits of network matrices

ALGORITHM

Step 1: Start the program.

Step 2: Declare the variable and get the number of nodes and branches.

Step 3: Get the node number and also check whether that node is connected to that branch.

Step 4: If it is connected, then check the direction of element, if it is towards the node put '-1' in branch path matrix else '1'.

Step 5**:** If it is not connected put '0' in the matrix.

Step 6: Print the branch path matrix.

Step 7: Calculate the number of branches in the matrix.

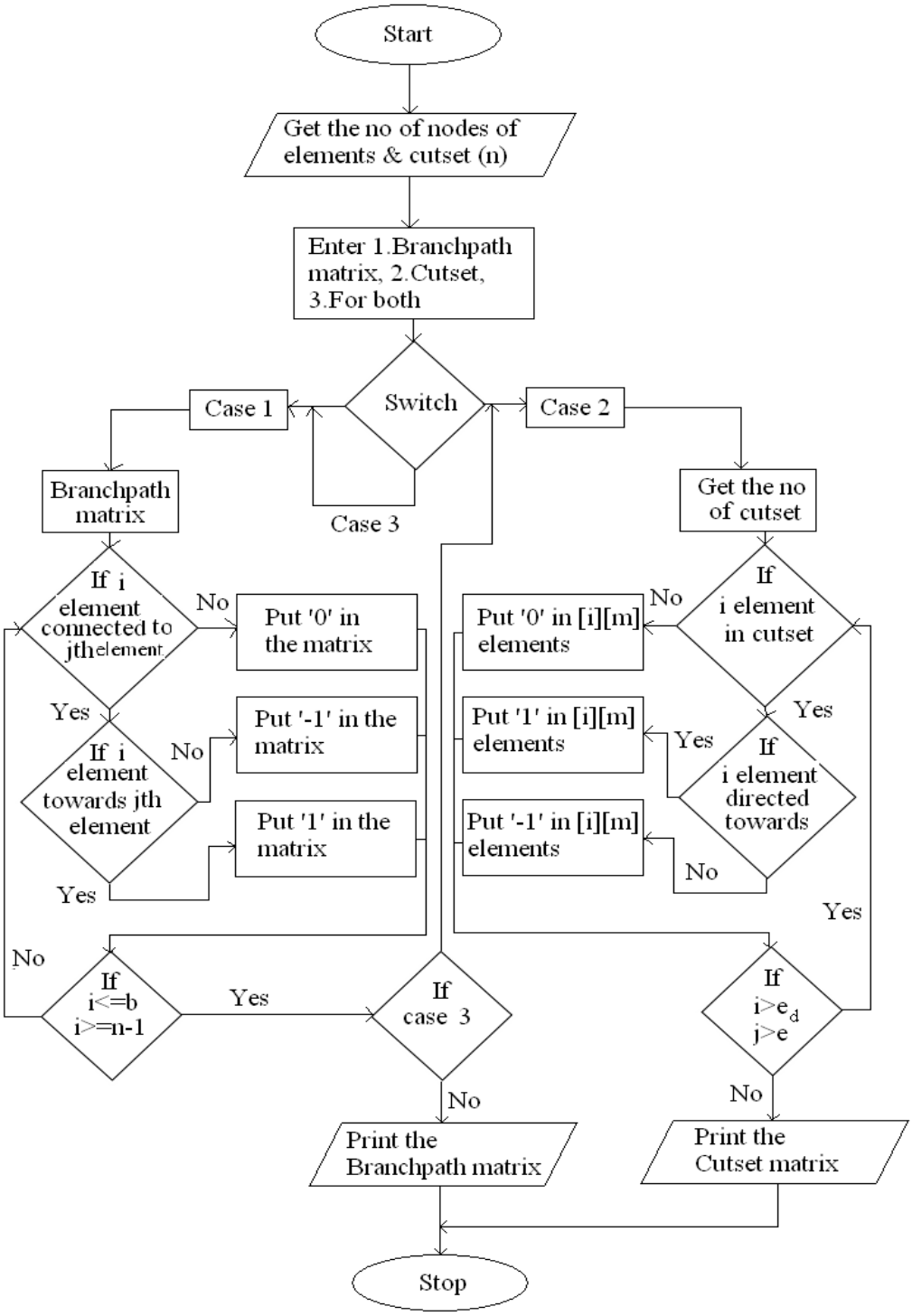
Step 8: Get the elements number & cutest number to check whether that element is present in the cutest or not.

Step 9: If the element is present check its direction & put '1', if it is opposite direction with the branch direction else put '1'.

Step 10: Print the cutest matrix.

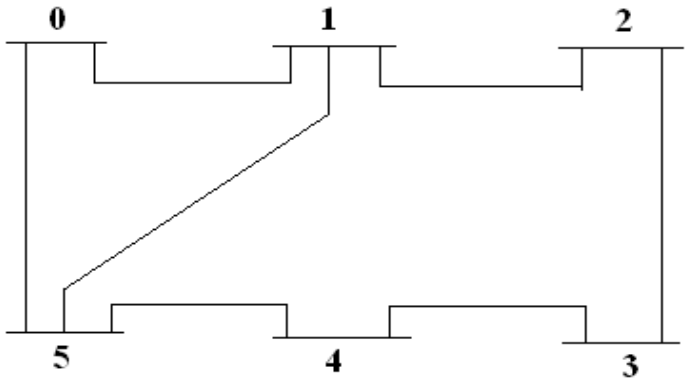
Step 11: Stop the program.

FLOW CHART


Start
Get the no of nodes of elements & cutset (n)
Enter 1.Branchpath matrix, 2.Cutset, 3.For both
Switch
Case 1
Case 2
Case 3
Branchpath matrix
Get the no of cutset
If i element connected to jth element
No
Put '0' in the matrix
Yes
If i element towards jth element
No
Put '-1' in the matrix
Put '1' in the matrix
Yes
No
If i<=b i>=n-1
Yes
If case 3
No
Print the Branchpath matrix
Stop
Put '0' in [i][m] elements
No
If i element in cutset
Yes
Put '1' in [i][m] elements
Yes
If i element directed towards
Put '-1' in [i][m] elements
No
Yes
If i>e_d j>e
No
Print the Cutset matrix

PROBLEM STATEMENT

Find the branch path matrix and cutset matrix of the given power system.



BRANCH PATH INCIDENCE MATRIX PROGRAM:

```
clc;
clear;

n = input('Enter number of nodes: ');
b = input('Enter number of branches: ');

k = zeros(b, n);  % Initialize incidence matrix

for i = 1:b
   for j = 1:n
      fprintf('\nIs node %d connected to branch %d?\n', j, i);
      p = input('Enter 1 for YES, 0 for NO: ');

      if p == 1
         fprintf('Is the direction of branch %d **towards** node %d?\n', i, j);
         q = input('Enter 1 for YES, 0 for NO: ');

         if q == 1
            k(i,j) = 1;
         else
            k(i,j) = -1;
         end
      else
         k(i,j) = 0;
      end
   end
end

% Display the branch-node incidence matrix
fprintf('\nThe Branch-Path Incidence Matrix is:\n');
```

```
for i = 1:b
   for j = 1:n
      fprintf('%3d\t', k(i,j));
   end
   fprintf('\n');
end
```

CUTSET MATRIX PROGRAM:

```
clc;
clear;

e = input('Enter the number of elements (branches): ');
c = input('Enter the number of cutsets: ');

a = zeros(e, c);  % Initialize the cutset matrix

for i = 1:e
   for j = 1:c
      fprintf('\nIs element %d present in cutset %d?\n', i, j);
      p = input('Enter 1 for YES, 0 for NO: ');

      if p == 1
         fprintf('Is element %d directed **along** the direction of cutset %d?\n', i, j);
         q = input('Enter 1 for YES, 0 for NO: ');

         if q == 1
            a(i, j) = 1;
         else
            a(i, j) = -1;
         end
      else
         a(i, j) = 0;
      end
   end
end

% Display the cutset matrix
fprintf('\nThe Cutset Matrix is:\n');
for i = 1:e
   for j = 1:c
      fprintf('%3d\t', a(i,j));
   end
   fprintf('\n');
end
```

OUTPUT:

VIVA QUESTIONS :

1. Define a cutset.
2. Indicate the number of cutset in a given network with ‘n’ nodes.
3. Differentiate tree and co tree.
4. List two applications of cutset matrix.
5. List two applications of branch path matrix.

STIMULATING QUESTIONS:

1. Justify the usage of matrices in network topology estimator.
2. Identify the number of branches, number of links, number of cutset in a network with ‘n’ nodes and ‘m’ elements.

RESULT:

Ex.No 5: SOLUTION OF POWER FLOW AND RELATED PROBLEMS USING GAUSS SEIDEL METHOD

AIM

To compute the program to find the load flow equations and solutions using Gauss Seidal method and Matlab.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. List the types of buses in a power system network.
2. Differentiate generator bus and load bus.
3. Indicate the importance of slack bus in a network.
4. Justify the need for load flow studies.

ALGORITHM

Step 1**:** Assume the all buses voltages are 1+j0 except at slack bus. The voltage at Slack bus is Constant.

Step 2**:** Assume a suitable value for specified change is bus voltages which is used to compare the actual changes in bus voltage.

Step 3**:** Set the value of k=0.

Step 4**:** Set the bus count p=1.

Step 5: Check for slack bus. If it is slack bus go to step 12. Otherwise go to step 6.

Step 6**:** Check for generator bus. If so go to next step or go to step 9.

Step 7**:** Set $V_p^k = V_{p(Spec)}^k$ and phase V_p^k as the K^{th} iteration.

Calculate the reactive power of generator as following equations.

$$\mathbf{Q_{p.cal}^{k+1}=(-1)*Im\{(V_p^k)^*[\sum_{q=1}^{p-1} Y_{pq}V_q^{k+1} + \sum_{q=p}^{n} Y_{pq}V_q^k]\}}$$

If $Q_{p.cal}^{k+1} < Q_{p.min}$ then $Q_p = Q_{p.min}$.

If $Q_{p.cal}^{k+1} > Q_{p.max}$ then $Qp = Q_{p.max}$.

Step 8**:** For Generator bus calculate the Bus voltage ($V_{p(temp)}^{k+1}$).

After calculating the Generator Bus Voltage go to step 11.

Step 9: For load bus calculate the value of V_p^{k+1} can be calculated.

Step 10: If acceleration factor is specified and modified the Bus voltage using the following equation.

$$\mathbf{V_{p(acc)}^{k+1} = V_p^k + \alpha(V_p^{k+1} - V_p^k).}$$

Then set $V_p^{k+1} = V_{p(acc)}^{k+1}$.

Step 11: Calculate the Change in bus voltage.

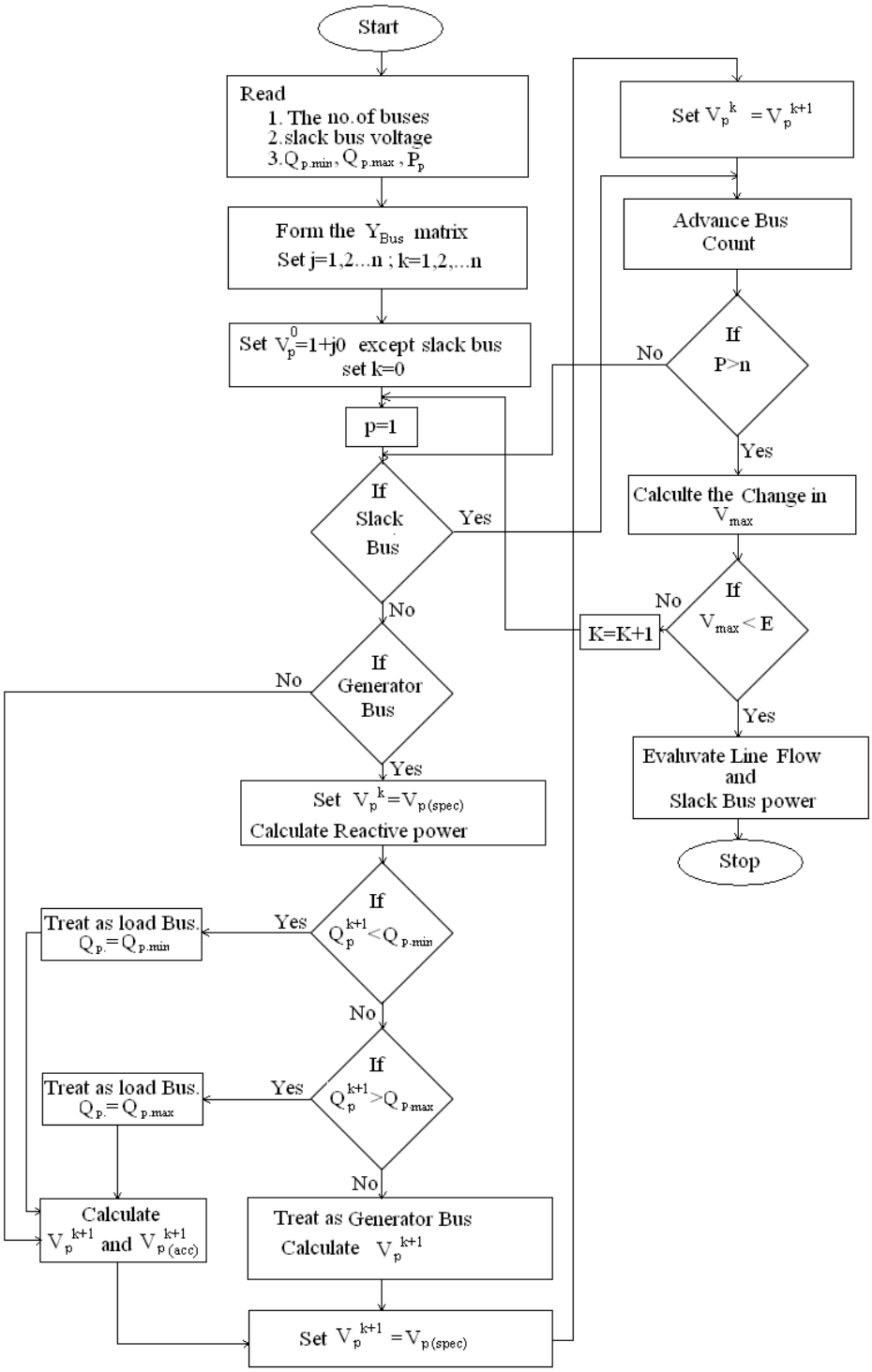
$$\mathbf{\Delta V_p^{k+1} = V_p^{k+1} - V_p^k.}$$

Step 12: Repeat 2 to 5 until all bus voltages are calculated.

Step 13: Find the largest among calculated voltage. If ΔV_{max} is less than prescribed value go to next step or increment iteration counts.

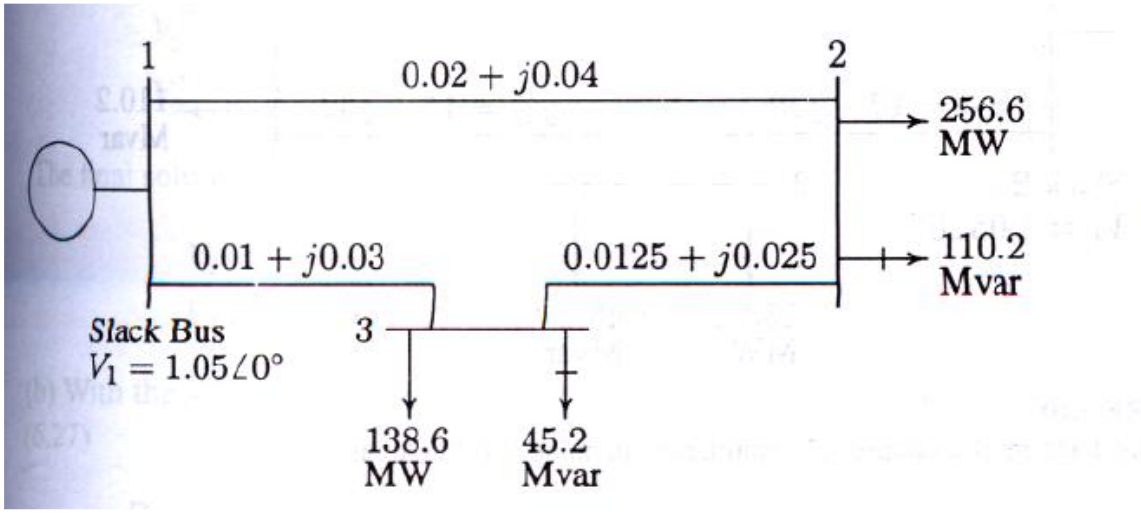
Step 14: Calculate line flows and slack bus power using bus voltages.

FLOW CHART


Start
Read
1. The no.of buses
2.slack bus voltage
3.$Q_{p.min}$, $Q_{p.max}$, P_p
Form the Y_{Bus} matrix
Set j=1,2...n ; k=1,2,...n
Set V_p^0=1+j0 except slack bus
set k=0
p=1
If Slack Bus
Yes
No
If Generator Bus
No
Yes
Set $V_p^k = V_{p(spec)}$
Calculate Reactive power
If $Q_p^{k+1} < Q_{p.min}$
Yes
Treat as load Bus.
$Q_p = Q_{p.min}$
No
If $Q_p^{k+1} > Q_{p.max}$
Yes
Treat as load Bus.
$Q_p = Q_{p.max}$
No
Calculate
V_p^{k+1} and $V_{p(acc)}^{k+1}$
Treat as Generator Bus
Calculate V_p^{k+1}
Set $V_p^{k+1} = V_{p(spec)}$
Set $V_p^k = V_p^{k+1}$
Advance Bus Count
If P>n
No
Yes
Calculte the Change in V_{max}
If $V_{max} < E$
No
K=K+1
Yes
Evaluvate Line Flow and Slack Bus power
Stop

PROBLEM STATEMENT

Figure shows the one-line diagram of a simple three-bus power system with generation at bus 1. The magnitude of voltage at bus 1 is adjusted to 1.05 per unit. The scheduled loads at buses 2 and 3 are as marked on the diagram. Line impedances are marked in per unit on a 100-MVA base and the line charging susceptances are neglected.



PROGRAM CODE:

```
function V = gauss_seidel_loadflow()

% Gauss-Seidel Load Flow for the given 3-bus system

% Ybus matrix
Z = [ 0    0.02+1i*0.04  0.01+1i*0.03;
    0.02+1i*0.04   0     0.0125+1i*0.025;
    0.01+1i*0.03  0.0125+1i*0.025   0 ];

n = 3;
```

```
Ybus = zeros(n,n);

% Build Ybus
for k = 1:3
    for m = 1:3
        if k ~= m && Z(k,m) ~= 0
            Ybus(k,k) = Ybus(k,k) + 1/Z(k,m);
            Ybus(k,m) = -1/Z(k,m);
        end
    end
end

% Known bus voltages
V = ones(n,1);
V(1) = 1.05 + 0j; % Slack bus with 0 angle

% Load data (PD, QD) in p.u.
P = [0; -2.566; -1.386]; % Negative since it is load
Q = [0; -1.102; -0.452];

% Convergence tolerance
tol = 1e-6;
max_iter = 100;

% Gauss-Seidel iteration
for iter = 1:max_iter
    V_old = V;
    for i = 2:n
        sumYV = 0;
        for j = 1:n
            if j ~= i
                sumYV = sumYV + Ybus(i,j)*V(j);
```

```
            end
        end
        S = P(i) + 1i*Q(i);
        V(i) = (1/Ybus(i,i)) * ((conj(S)/conj(V(i))) - sumYV);
    end
    if max(abs(V - V_old)) < tol
        fprintf('Gauss-Seidel converged in %d iterations\n', iter);
        break;
    end
end

% Output
fprintf('\nBus Voltages (magnitude and angle):\n');
for i = 1:n
    fprintf('Bus %d: |V| = %.4f, Angle = %.4f degrees\n', i, abs(V(i)), angle(V(i))*180/pi);
end
```

OUTPUT:

VIVA QUESTIONS :

1. Define acceleration factor in Gauss Seidel method.
2. Indicate the voltage equation to be iterated in Gauss Seidel method.
3. Comment on the memory requirement of Gauss Seidal algorithm.

STIMULATING QUESTIONS:

1. Suggest a way to increase the convergence accuracy in Gauss Seidel method.
2. Indicate the drawbacks of solving load flow problem using Gauss Seidel method

RESULT

Ex.No 6: SOLUTION OF POWER FLOW AND RELATED PROBLEMS USING NEWTON RAPHSON METHOD

AIM

To compute the load flow equations for a multibus system using Newton Raphson method.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. Differentiate DC and AC load flow methods.
2. Recall the advantages of using NR method in solving non linear problems.
3. Indicate the constraints to be satisfied while solving a network load flow.

ALGORITHM

Step 1: Assume the flat voltage for all buses except the slack bus and gen.bus.

Step 2: Set convergence criterion 'E' if the largest or the absolute of the residue is less than 'E' than The solution gets converged and the program gets terminated after calculating line flows, Otherwise repeat.

Step 3: Set the iteration count iter=1.

Step 4: Set the bus count n=1.

Step 5: Check if the bus is a slack bus if so goes to step 10.

Step 6: Calculate the real and reactive power Pp and Qp.

Step 7: Evaluate $\Delta P_p^k = P_{sp} - P_{pcal}$.

Step 8: Check if the bus is a generator bus, if so compare Q_p^k whether is there within the limits, violated and find the reactive power and treat bus as load bus if not calculate the voltage residue.

$$|\Delta V_p|^2=|V_{p,}S_p|^2-|V_{p,c}|^2$$

and go to step 10.

Step 9: Evaluate $\Delta Q_p^k=Q_{sp}-Q_{p,cal}$

Step 10: Advance bus count by 1 and check whether all the buses are counted if not go to step 8.

Step 11: Determine the largest absolute value or residue

Step 12: If largest of the residue is less than 'E' go to step 17.

Step 13: Find Jacobian matrix.

Step 14: Calculate new bus voltage

$$e_p^{k+1}=e_p^k+\Delta e_p^k$$

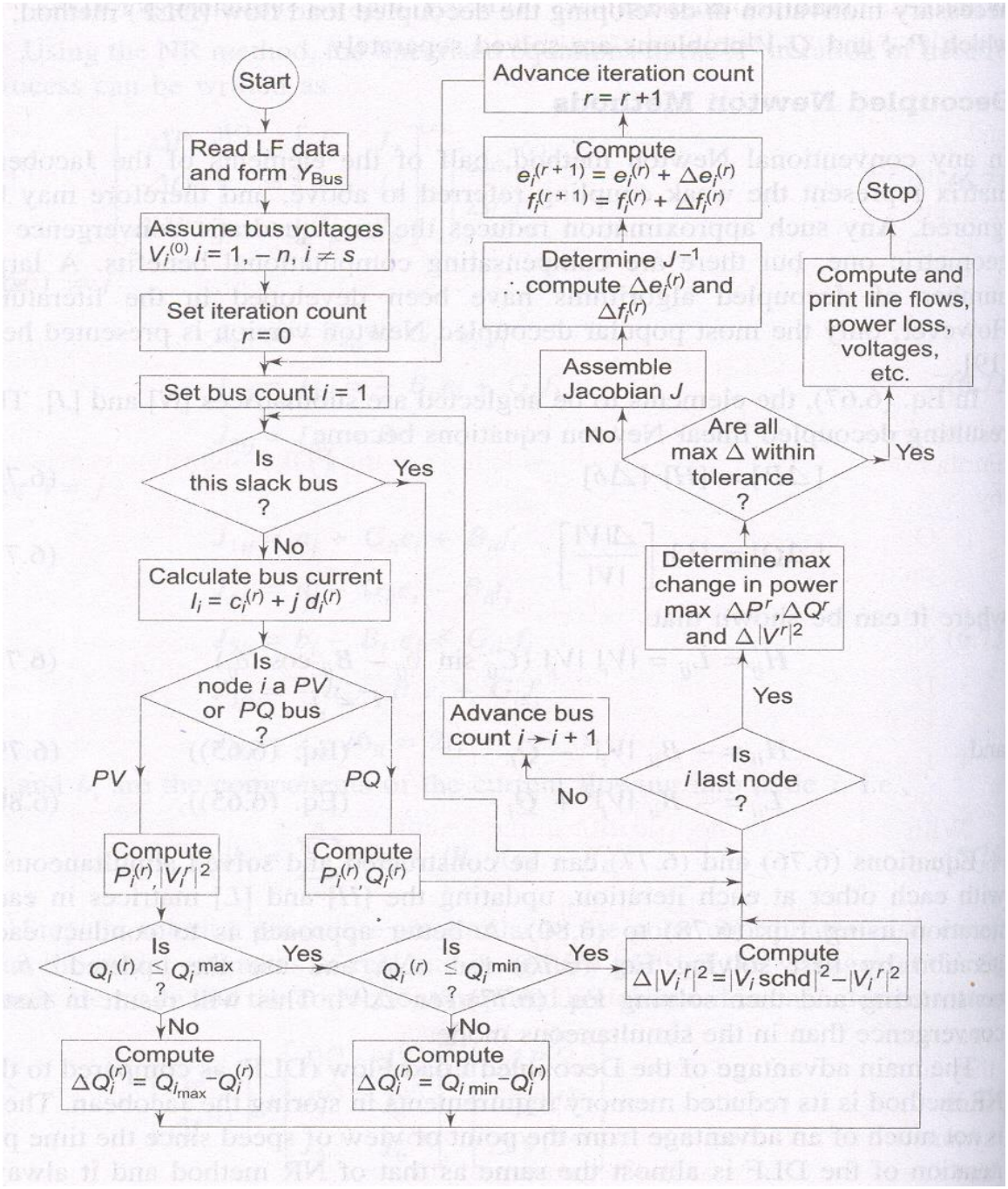
$$f_p^{k+1}=f_p^k+\Delta f_p^k$$

Step 15: Advance iteration count by 1& go to step 4.

Step 16: Evaluate bus & line voltage

Step 17: Print the result

FLOWCHART


Start
Read LF data and form Y_{Bus}
Assume bus voltages $V_i^{(0)}$ $i = 1, -n, i \neq s$
Set iteration count $r = 0$
Set bus count $i = 1$
Is this slack bus ?
Yes
No
Calculate bus current $I_i = c_i^{(r)} + j\, d_i^{(r)}$
Is node i a PV or PQ bus ?
PV
PQ
Compute $P_i^{(r)}, |V_i^r|^2$
Compute $P_i^{(r)}, Q_i^{(r)}$
Is $Q_i^{(r)} \leq Q_{i\max}$?
Yes
No
Compute $\Delta Q_i^{(r)} = Q_{i_{max}} - Q_i^{(r)}$
Is $Q_i^{(r)} \leq Q_{i\min}$?
Yes
No
Compute $\Delta Q_i^{(r)} = Q_{i\min} - Q_i^{(r)}$
Compute $\Delta |V_i^r|^2 = |V_i \text{ schd}|^2 - |V_i^r|^2$
Is i last node ?
No
Yes
Advance bus count $i \rightarrow i + 1$
Determine max change in power max $\Delta P^r, \Delta Q^r$ and $\Delta |V^r|^2$
Are all max Δ within tolerance ?
No
Yes
Assemble Jacobian J
Determine J^{-1} $\therefore$ compute $\Delta e_i^{(r)}$ and $\Delta f_i^{(r)}$
Compute $e_i^{(r+1)} = e_i^{(r)} + \Delta e_i^{(r)}$ $f_i^{(r+1)} = f_i^{(r)} + \Delta f_i^{(r)}$
Advance iteration count $r = r + 1$
Stop
Compute and print line flows, power loss, voltages, etc.

PROBLEM STATEMENT

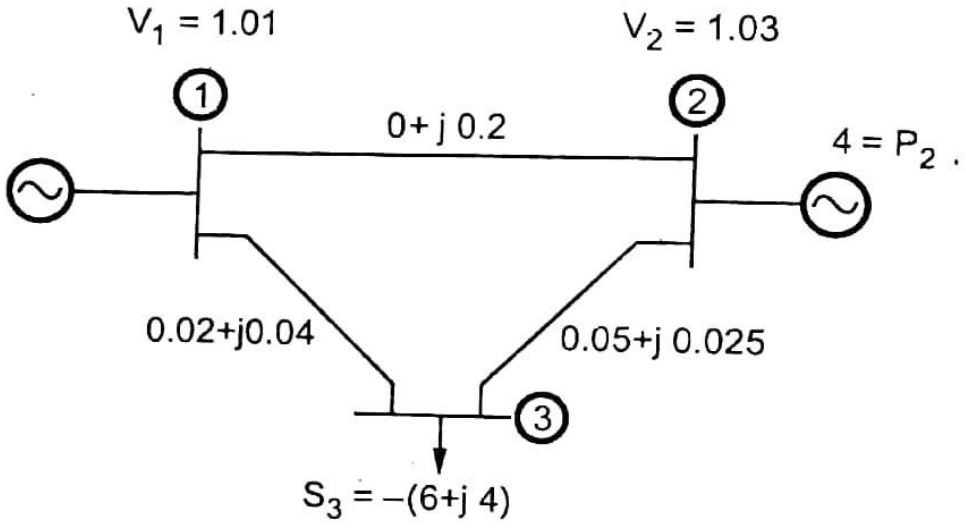
Bus data in p.u.

Bus Number	Type	Generator		Load		Voltage	Angle	Reactive Power Limit	
		P_G	Q_G	P_D	Q_D			Q_{min}	Q_{max}
1	Slack	0	0	0	0	1.01	0	0	0
2	PV	4.0	0	0	0	1.03	0	-2	2
3	PQ	0	0	6	4.0	1.0	0	0	0

Line data in p.u.

From	To	R p.u.	X p.u.	½ B1 p.u.	Tap Position
1	2	0	0.2	0	1
2	3	0.02	0.04	0	1
2	3	0.05	0.025	0	1

Base MVA: 100MVA



PROGRAM CODE:

```
% Newton-Raphson Load Flow for the given 3-bus system

clc; clear;

%% Bus Types:
% 1 - Slack
% 2 - PV
% 3 - PQ

% System base
baseMVA = 100;

% Number of buses
nb = 3;

% Bus type: 1-Slack, 2-PV, 3-PQ
bus_type = [1; 2; 3];

% Given power data (in per unit)
P = [0; -4.0; -6.0];   % PG - PD in pu
Q = [0;  0.0; -4.0];   % QG - QD in pu (initial guess for PV)

% Initial voltage magnitude and angle (in radians)
V = [1.01; 1.03; 1.00];
delta = [0; 0; 0];

% Admittance Matrix (Ybus)
Z = zeros(nb, nb);
Y = zeros(nb);
```

```
% Line admittances (in pu)
Y(1,2) = -1j*5;              % 1-2 line: 0+j0.2
Y(2,3) = -1/(0.02+1j*0.04);   % 2-3 line
Y(2,3) = Y(2,3) + (-1/(0.05+1j*0.025)); % Parallel line 2-3
Y(1,3) = 0;  % No direct connection

% Off-diagonal elements
Y(2,1) = Y(1,2);
Y(3,2) = Y(2,3);

% Diagonal elements
for i = 1:nb
   for j = 1:nb
      if i ~= j
         Y(i,i) = Y(i,i) - Y(i,j);
      end
   end
end

% Start NR iteration
max_iter = 10;
epsilon = 1e-6;

PQ_idx = find(bus_type == 3);
PV_idx = find(bus_type == 2);

for iter = 1:max_iter
   % Calculate P and Q from current voltages
   P_calc = zeros(nb,1);
   Q_calc = zeros(nb,1);
```

```
    for i = 1:nb
        for k = 1:nb
            P_calc(i) = P_calc(i) + V(i)*V(k)*(real(Y(i,k))*cos(delta(i)-delta(k)) +
imag(Y(i,k))*sin(delta(i)-delta(k)));
            Q_calc(i) = Q_calc(i) + V(i)*V(k)*(real(Y(i,k))*sin(delta(i)-delta(k)) -
imag(Y(i,k))*cos(delta(i)-delta(k)));
        end
    end

    dP = P(2:3) - P_calc(2:3);
    dQ = Q(3) - Q_calc(3);
    mismatch = [dP; dQ];

    if max(abs(mismatch)) < epsilon
        break;
    end

    % Jacobian submatrices (only 2 unknowns in this case)
    J11 = zeros(2);
    J22 = zeros(1);
    J12 = zeros(2,1);
    J21 = zeros(1,2);

    for i = 2:3
        for k = 2:3
            if i == k
                for m = 1:nb
                    J11(i-1,k-1) = J11(i-1,k-1) + V(i)*V(m)*(-real(Y(i,m))*sin(delta(i)-delta(m)) +
imag(Y(i,m))*cos(delta(i)-delta(m)));
                end
                J11(i-1,k-1) = J11(i-1,k-1) - V(i)^2 * imag(Y(i,i));
            else
```

```
            J11(i-1,k-1) = V(i)*V(k)*(real(Y(i,k))*sin(delta(i)-delta(k)) - imag(Y(i,k))*cos(delta(i)-
delta(k)));
         end
      end
   end

   for i = 3
      for k = 3
         if i == k
            for m = 1:nb
               J22(i-2,k-2) = J22(i-2,k-2) + V(i)*(real(Y(i,m))*cos(delta(i)-delta(m)) +
imag(Y(i,m))*sin(delta(i)-delta(m)));
            end
            J22(i-2,k-2) = J22(i-2,k-2) + V(i)*real(Y(i,i));
         else
            J22(i-2,k-2) = V(i)*(-real(Y(i,k))*cos(delta(i)-delta(k)) - imag(Y(i,k))*sin(delta(i)-
delta(k)));
         end
      end
   end

   for i = 2:3
      J12(i-1) = V(i)*real(Y(i,3))*cos(delta(i)-delta(3)) + imag(Y(i,3))*sin(delta(i)-delta(3));
   end

   for i = 3
      for k = 2:3
         J21(1,k-1) = V(i)*(real(Y(i,k))*cos(delta(i)-delta(k)) + imag(Y(i,k))*sin(delta(i)-delta(k)));
      end
   end

   % Construct Jacobian
```

```
    J = [J11 J12; J21 J22];

    % Solve for corrections
    dx = J\mismatch;

    delta(2:3) = delta(2:3) + dx(1:2);
    V(3) = V(3) + dx(3);
end

% Print Results
fprintf('Converged in %d iterations\n', iter);
for i = 1:nb
    fprintf('Bus %d: V = %.4f p.u., Angle = %.4f deg\n', i, V(i), rad2deg(delta(i)));
end
```

OUTPUT:

VIVA QUESTIONS:

1. Recall the elements of Jacobian 1 and Jacobian 3 in iterative equation.
2. Find the elements of Jacobian 4 for a network with one generator bus and three load buses apart from a slack bus.
3. Indicate the relationship between frequency and rotor displacement angle in an alternator.

STIMULATING QUESTIONS:

1. Compare the number of iterations required for convergence in Gauss Seidel and Newton

Raphson method.

2. Compare the accuracy of Gauss Seidel and Newton Raphson method.

RESULT

Ex.No 7: SOLUTION OF POWER FLOW AND RELATED PROBLEMS USING FAST DECOUPLED LOAD FLOW METHOD

AIM

To compute the program to find the load flow equations and solutions using FDLF method.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. Recall the control loops available in a synchronous generator.
2. State the relationship between reactive power delivered and terminal voltage of an alternator.
3. Differentiate generator bus and slack bus

ALGORITHM

Step 1: Read the system data.

Step 2: Initialize all the bus voltages.

Step 3: Form admittance matrix.

Step 4: Form B', B" and then invert them.

Step 5: Compute $\Delta P = P^{spec} - P^{calc}$

Step 6: Check for convergence. If converged, set $P_{conv} = 1$, and go to step 10.

Step 7: Form Jacobian matrix H.

Step 8: If not converged, find $[\Delta\delta]$.

Step 9: Update δ value.

Step 10: Compute ΔQ.

Step 11: Check for convergence. If converged set $Q_{conv} = 1$, and go to step 15.

Step 12: Form [L].

Step 13: If not converged, find $[\Delta V]$.

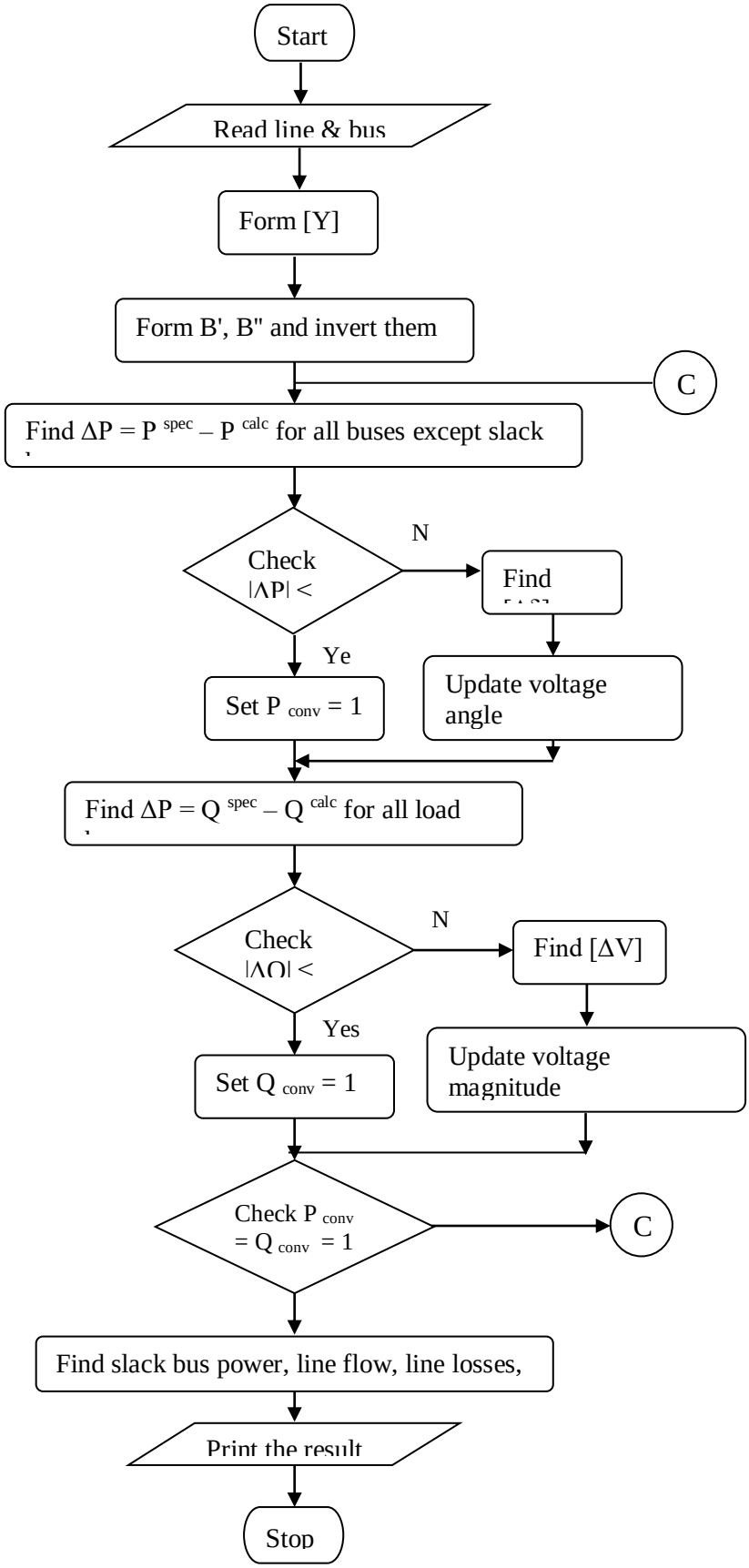
Step 14: Update |V| value.

Step 15: If $P_{conv} = 1$ and $Q_{conv} = 1$, then solution is obtained and print the result.

Step 16: Else go to step 5.

Step 17: Stop.

FLOWCHART


Start
Read line & bus
Form [Y]
Form B', B" and invert them
C
Find ΔP = P spec – P calc for all buses except slack
Check
N
Find
Ye
Set P conv = 1
Update voltage angle
Find ΔP = Q spec – Q calc for all load
Check
N
Find [ΔV]
Yes
Set Q conv = 1
Update voltage magnitude
Check P conv = Q conv = 1
C
Find slack bus power, line flow, line losses,
Print the result
Stop

PROBLEM STATEMENT

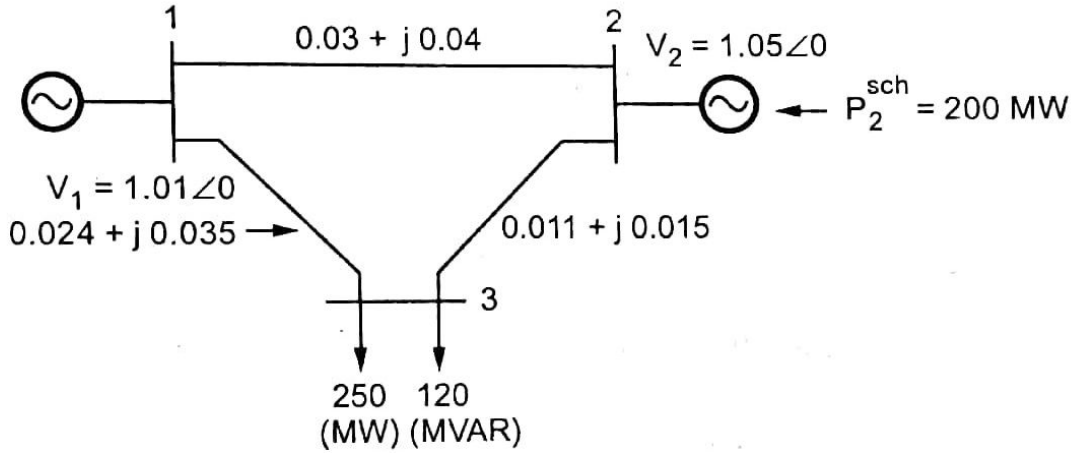
Bus data in p.u.

Bus Number	Type	Generator (MW)		Load (MW)		Voltage	Angle	Reactive Power Limit	
		P_G	Q_G	P_D	Q_D			Q_{min}	Q_{max}
1	Slack	0	0	0	0	1.06	0	0	0
2	PQ	0	0	270	130	1.0	0	0	0
3	PV	230	0	0	0	1.04	0	0	230

Line data in p.u.

From	To	R p.u.	X p.u.	½ B1 p.u.	Tap Position
1	2	0.03	0.04	0	1
2	3	0.011	0.015	0	1
1	3	0.024	0.035	0	1

Base MVA: 100MVA



PROGRAM CODE:

```
function [V, delta] = fdlf_solver(bus_data, line_data, baseMVA)
% Fast Decoupled Load Flow Solver
% Inputs:
%   bus_data: [bus_no type PG QG PD QD V Qmin Qmax]
%   line_data: [From To R X B/2 Tap]
%   baseMVA: system base power (e.g., 100)
% Outputs:
%   V: bus voltage magnitudes
%   delta: bus voltage angles in degrees

n_bus = size(bus_data, 1);
bus_type = bus_data(:, 2);  % 1: Slack, 2: PV, 3: PQ
slack = find(bus_type == 1);
PV = find(bus_type == 3);
PQ = find(bus_type == 2);

%% Construct Ybus
Ybus = zeros(n_bus);
for k = 1:size(line_data,1)
    from = line_data(k,1);
    to   = line_data(k,2);
    R    = line_data(k,3);
    X    = line_data(k,4);
    B    = line_data(k,5);
    a    = line_data(k,6);

    Z = R + 1i*X;
    y = 1/Z;
    b_shunt = 1i * B;
```

```
    Ybus(from, from) = Ybus(from, from) + y/(a^2) + b_shunt;
    Ybus(to, to)     = Ybus(to, to)     + y + b_shunt;
    Ybus(from, to)   = Ybus(from, to)   - y/a;
    Ybus(to, from)   = Ybus(from, to);
end

%% Initialization
V = bus_data(:,7); % Initial voltage magnitudes
delta = zeros(n_bus,1); % Angles in radians

P = (bus_data(:,3) - bus_data(:,5)) / baseMVA; % PG - PD
Q = (bus_data(:,4) - bus_data(:,6)) / baseMVA; % QG - QD

tol = 1e-6;
max_iter = 20;

Bp = imag(Ybus);
Bpp = Bp;

Bp(slack,:) = []; Bp(:,slack) = [];
Bpp([slack; PV],:) = []; Bpp(:,[slack; PV]) = [];

%% Iterative Solution
for iter = 1:max_iter
    % Calculate Power
    for i = 1:n_bus
        Pi = 0; Qi = 0;
        for k = 1:n_bus
            Y = Ybus(i,k);
            Pi = Pi + V(i)*V(k)*( real(Y)*cos(delta(i)-delta(k)) + imag(Y)*sin(delta(i)-delta(k)) );
            Qi = Qi + V(i)*V(k)*( real(Y)*sin(delta(i)-delta(k)) - imag(Y)*cos(delta(i)-delta(k)) );
        end
```

```
    P_calc(i,1) = Pi;
    Q_calc(i,1) = Qi;
end

dP = P - P_calc;
dQ = Q - Q_calc;

dP(slack) = [];
dQ([slack; PV]) = [];

d_delta = Bp \ dP;

k = 1;
for i = 1:n_bus
    if i ~= slack
        delta(i) = delta(i) + d_delta(k);
        k = k + 1;
    end
end

dV = Bpp \ dQ;
k = 1;
for i = 1:n_bus
    if ismember(i, PQ)
        V(i) = V(i) + dV(k);
        k = k + 1;
    end
end

if max(abs(dP)) < tol && max(abs(dQ)) < tol
    fprintf('FDLF Converged in %d iterations.\n', iter);
    break;
```

```
    end
end

delta = rad2deg(delta); % Convert to degrees
end

% --- bus_data: [BusNo Type PG QG PD QD V Qmin Qmax]
bus_data = [
    1 1 0     0   0    0    1.06 0   0;
    2 2 0     0 270  130    1.00 0   0;
    3 3 230   0   0    0    1.04 0 230
];



% --- line_data: [From To R X B/2 Tap]
line_data = [
    1 2 0.03  0.04  0  1;
    2 3 0.011 0.015 0  1;
    1 3 0.024 0.035 0  1
];

baseMVA = 100;

[V, delta] = fdlf_solver(bus_data, line_data, baseMVA);

disp('Bus Voltages and Angles:');
disp(table((1:3)', V, delta, 'VariableNames', {'Bus', 'Voltage_pu', 'Angle_deg'}));
```

OUTPUT:

VIVA QUESTIONS:

1. Indicate the Jacobian J2 and J3 of a network 1 slack bus and 1 generator bus and 2 load buses using FDLF method
2. Indicate the reason for Jacobian sparsity in FDLF method.

STIMULATING QUESTIONS

1. Establish the relationship between real power and voltage in an alternator during transient disturbance

RESULT

Ex.No-8 **SHORT CIRCUIT ANALYSIS**

Date:

AIM

To obtain the short circuit analysis for a IEEE 14 bus system.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

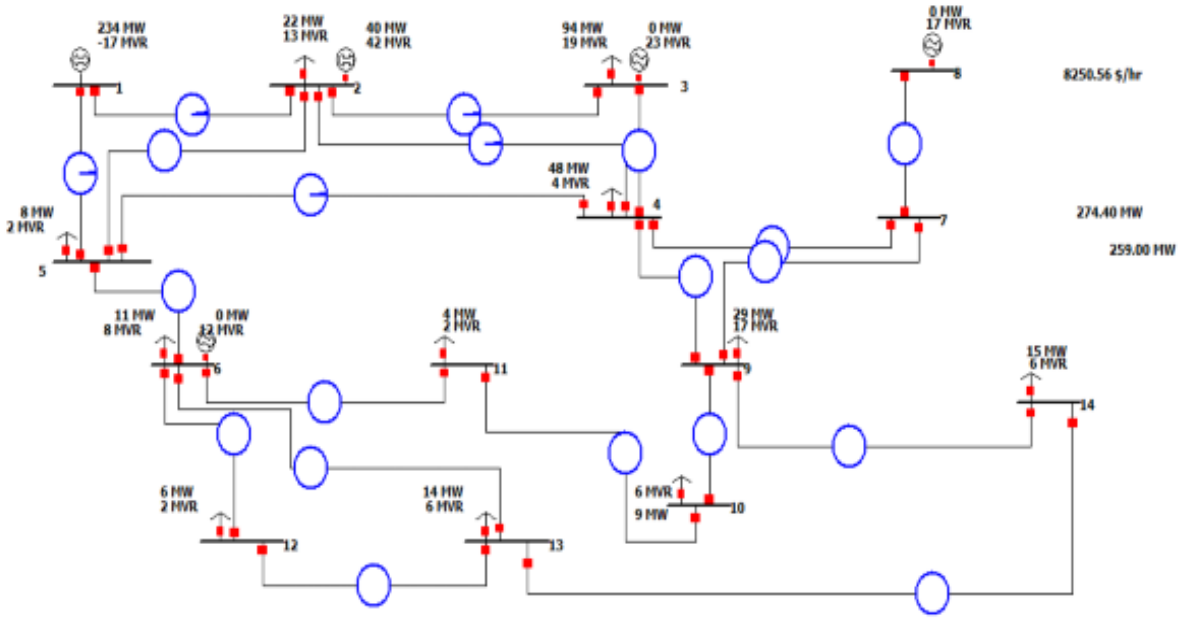
PREREQUISITE QUESTIONS

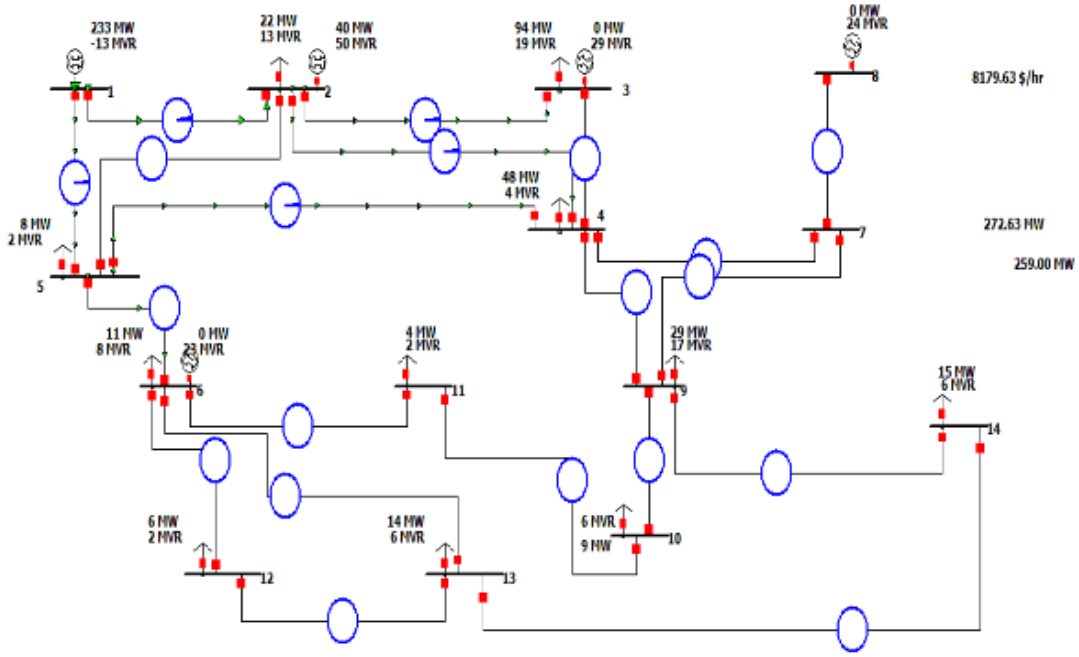
1. Classify the power system faults based on voltage symmetry
2. Indicate the magnitude of current flowing through the neutral conductor during three phase fault.
3. Differentiate symmetrical and unsymmetrical faults based on fault severity.

Procedure :

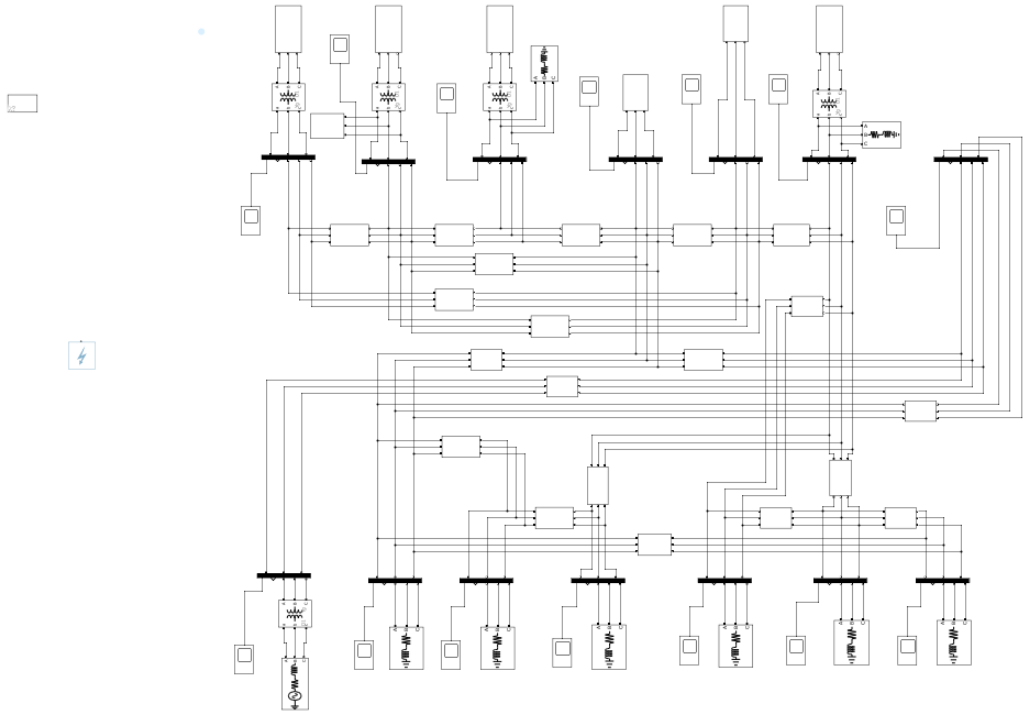
- Create a new file in edit mode by selecting File - New File.
- Browse the components and build the bus system.
- Execute the program in run mode by selecting tools-fault analysis.
- Select the fault on which bus and calculate.
- Tabulate the results.

14 bus system :





SIMULINK MODEL:



OUTPUT:

VIVA QUESTIONS:

1. Indicate the equation to determine short circuit capacity at any point in a network.
2. Justify the need to determine fault current.
3. State the application of short circuit analysis.

STIMULATING QUESTIONS:

1. Differentiate line to line and line to ground fault based on probability of occurrence.
2. What will happen if a light bulb is connected between phase and earth.

RESULT:

Ex.No-9: ECONOMIC DISPATCH IN POWER SYSTEMS

AIM

To write a MATLAB program to compute the economic dispatch control by iteration method.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

4. Differentiate Economic load dispatch and unit commitment.
5. List any 3 constraints in unit commitment problem.
6. Mention the stages of load dispatching in a power system

ALGORITHM

Step1: Start the program.

Step2: Set the initial value such that TFRs co-efficient should be less than the Chosen by us.

Step3: Calculate the power Pload for load=1, 2, 3….n

Where,

n- The number of plants connected to the load.

Step4: Calculate the tolerance value by using the formula.

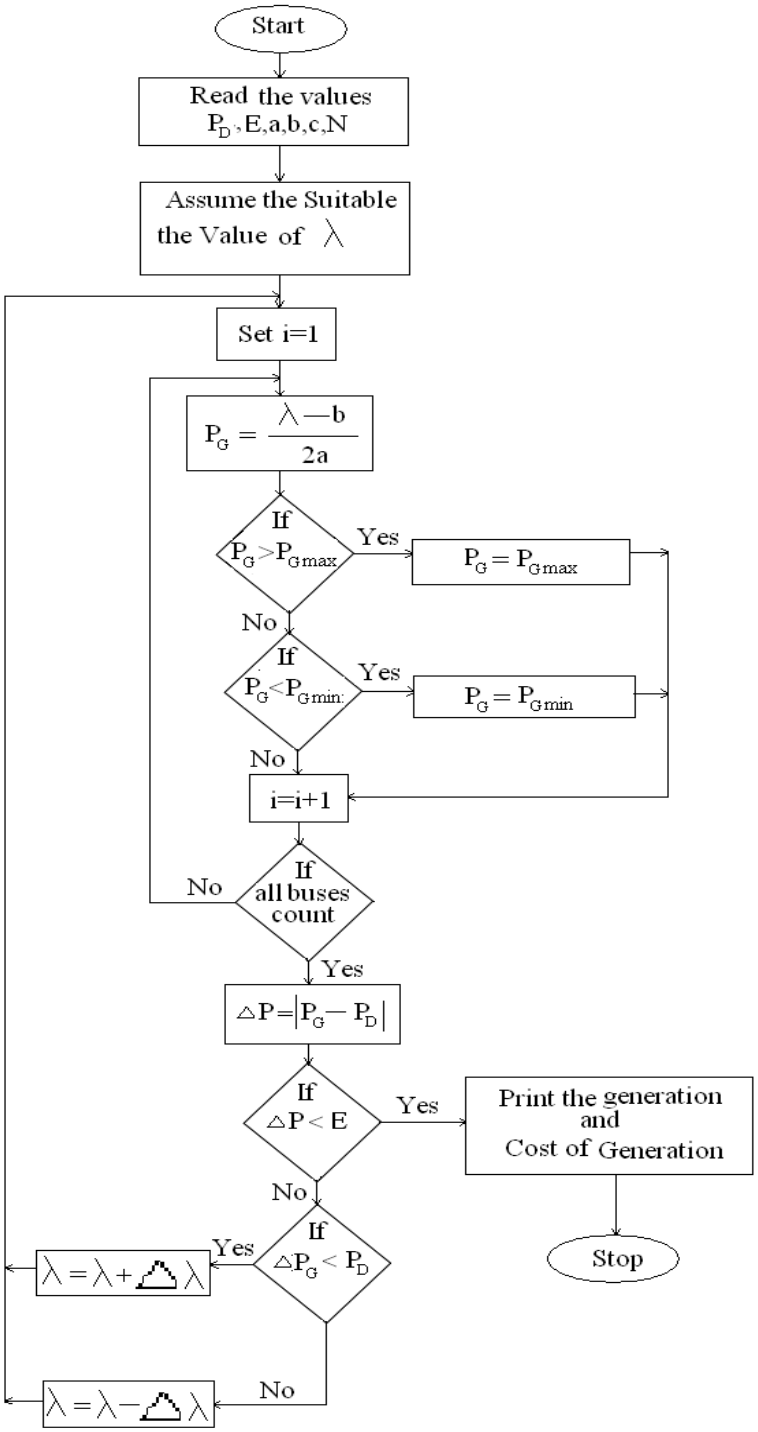
$$\varepsilon = P_r - \sum_{load=1}^{n} P_{load}$$

Step5: If the tolerance values negligible stop the program otherwise continue the Iteration.

Step6: Print the result.

Step7: Stop the program.

FLOWCHART



PROGRAM CODE:

```
clc;
clear;

% Input: Generator cost coefficients a + b*P + c*P^2
% Format: [a b c Pmin Pmax]
gen_data = [...
    500 5.3 0.004 100 600;   % Generator 1
    400 5.5 0.006 100 400;   % Generator 2
    200 5.8 0.009 50  200];  % Generator 3

Pd = 850;  % Total demand (MW)
n = size(gen_data, 1);  % Number of generators

% Tolerance and initialization
tol = 0.01;
lambda = 6;  % Initial guess
delta = 1;

while abs(delta) > tol
    P = zeros(n, 1);  % Power output for each generator
    dPd = 0;          % Power mismatch

    for i = 1:n
        a = gen_data(i, 1);
        b = gen_data(i, 2);
        c = gen_data(i, 3);
        P(i) = (lambda - b) / (2 * c);

        % Check generator limits
```

```
        if P(i) > gen_data(i, 5)
            P(i) = gen_data(i, 5);
        elseif P(i) < gen_data(i, 4)
            P(i) = gen_data(i, 4);
        end
    end

    P_total = sum(P);
    dPd = Pd - P_total;

    % Derivative sum for delta lambda
    dL = sum(1 ./ (2 * gen_data(:, 3)));

    % Update lambda
    lambda = lambda + dPd / dL;

    delta = dPd;  % Mismatch to check convergence
end

% Calculate total cost
total_cost = 0;
for i = 1:n
    a = gen_data(i, 1);
    b = gen_data(i, 2);
    c = gen_data(i, 3);
    total_cost = total_cost + (a + b * P(i) + c * P(i)^2);
end

% Output results
fprintf('Economic Dispatch Results:\n');
for i = 1:n
    fprintf('Generator %d: %.2f MW\n', i, P(i));
```

```
end
fprintf('Total Generation: %.2f MW\n', sum(P));
fprintf('Total Cost: ₹%.2f\n', total_cost);
```

OUTPUT:

VIVA QUESTIONS:

1. List the constraints in Economic load dispatch.
2. State the Khun-Tucker condition for optimality.
3. Differentiate local minima and global minima in an Economic Load Dispatch problem
4. List the inequality constraints to be satisfied while solving load flow problem

STIMULATING QUESTIONS:

3. Examine the significance of Lambda in Economic load dispatch problem.
4. Determine the coordination equation for a system with power losses.

RESULT:

Ex.No-10: TRANSIENT STABILITY ANALYSIS

AIM

To analyze the modeling aspects of synchronous machine and Transient stability of Multimachine power systems.

SOFTWARE REQUIRED

MATLAB/ETAP/Power world simulator

PREREQUISITE QUESTIONS

1. Define stability
2. Recall the equation to estimate power transferred between any two nodes in a networks.
3. Differentiate steady state and transient state stability.
4. State the characteristics of an infinite bus used for stability estimation.

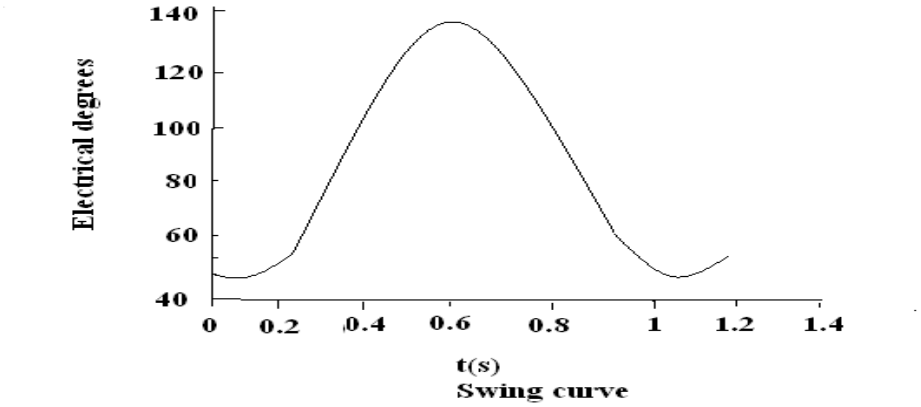
ASSUMPTIONS

i. Generator saliency neglected is $X_d=X_q$;$X_d'=X_q'$

ii. All resistances are neglected.

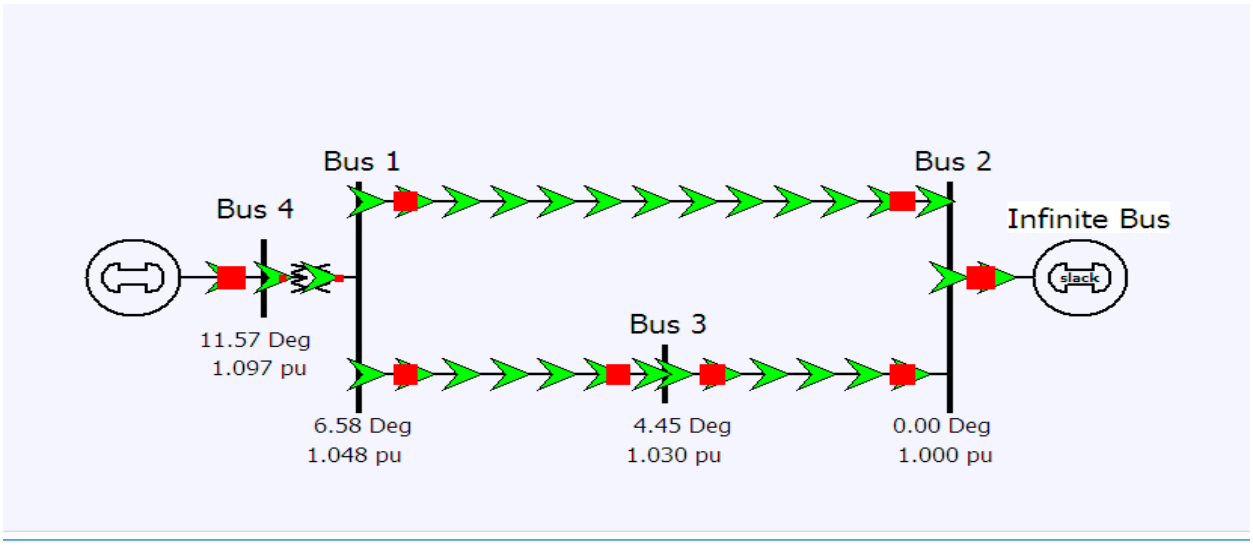
iii.Damping is neglected

iv. Generators are represented by internal voltage behind the transient reactance.



MULTIMACHINE TRANSIENT STABILITY – SAMPLE PROBLEM

Consider a 60 HZ machine for which H=2.7MJ/MVA and it is initially operating in steady state with input and output of 1 p.u. and an angular displacement of 45 electrical degree with respect to an infinite bus bar. Upon occurance of a fault, assume that the input remains constant and the output is given by $P_e = \delta/90^o$. Calculate and plot the swing curve by the step-by-step method ii. Using the time interval $\Delta t=0.05$ s.up to t=1 s.step-by-step method –ll using the time interval $\Delta t=0.05$ s and upto t=1s.



PROGRAM CODE:

```
clc;
clear;
% Parameters
H = 2.7;                  % Inertia constant (MJ/MVA)
f = 60;                   % System frequency (Hz)
omega_s = 2 * pi * f;        % Synchronous speed (rad/s)
Pm = 1.0;                 % Mechanical input power (p.u.)
delta0_deg = 45;            % Initial delta in degrees
delta0 = deg2rad(delta0_deg); % Convert to radians
t_max = 1.0;              % Max simulation time (s)
dt = 0.05;                % Time step (s)
n = t_max/dt;             % Number of steps

% Initialize arrays
delta = zeros(1, n+1);       % Rotor angle delta in radians
omega = zeros(1, n+1);       % Angular velocity deviation
time = 0:dt:t_max;          % Time vector

% Initial conditions
delta(1) = delta0;
omega(1) = 0;               % System initially in steady state

% Constant
M = (2 * H) / omega_s;       % Moment of inertia

% Time stepping using Step-by-Step Method II
for i = 1:n
    Pe = delta(i) / (pi/2);              % Electrical power output
    accel = (Pm - Pe) / M;               % Acceleration (rad/s^2)

    % Update omega and delta
```

```
    omega(i+1) = omega(i) + accel * dt;      % ω at i+1
    delta(i+1) = delta(i) + omega(i+1) * dt; % δ at i+1 using updated ω
end

% Convert delta to degrees for plotting
delta_deg = rad2deg(delta);

% Plot swing curve
figure;
plot(time, delta_deg, 'b-o', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Rotor Angle δ (degrees)');
title('Swing Curve using Step-by-Step Method II');
grid on;
```

OUTPUT:

VIVA QUESTIONS:

1. Define critical clearing angle.
2. Interpret the stability of a system from a curve plotted between rotor displacement angle and time.
3. State the significance of power angle curve
4. Indicate the limits for δ in a stable system.

STIMULATING QUESTIONS:

1. What will happen if a generator with rated capacity 2000MW is subjected to a sudden load addition of 1500MW?

RESULT

EX. NO.: 11 LOAD – FREQUENCY DYNAMICS OF SINGLE- AREA POWER SYSTEMS

AIM: To become familiar with the modeling and analysis of load frequency and tie line flow dynamics of a power systems with load frequency controller (LFC)

SOFTWARE REQUIRED: MATLAB

THEORY:

Primary control: The speed change from the synchronous speed initiates the governor control action resulting in all the participating generator-turbine units taking up the change in load, and stabilizes the system frequency.

Secondary control: It adjusts the load reference set points of selected turbine-generator units so as to give nominal value of frequency. The frequency control is a matter of speed control of the machines in the generating stations. The frequency of a power system is dependent entirely upon the speed in which the generators are rotated by their prime movers. All prime movers, whether they are steam or hydraulic turbines, are equipped with speed governors which are purely mechanical speed sensitive devices, to adjust the gate or control valve opening for the constant speed.

PROCEDURE:

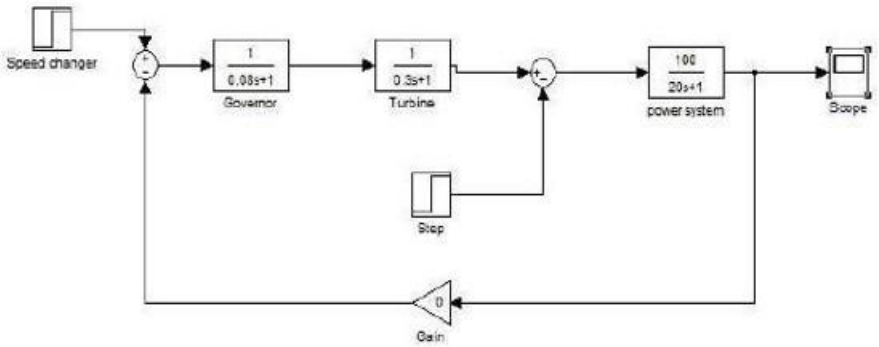
1. Enter the command window of the MATLAB.
2. Create a new Model by selecting File - New – Model.
3. Pick up the blocks from the simulink library browser and form a block diagram.
4. After forming the block diagram, save the block diagram.
5. Double click the scope and view the result.

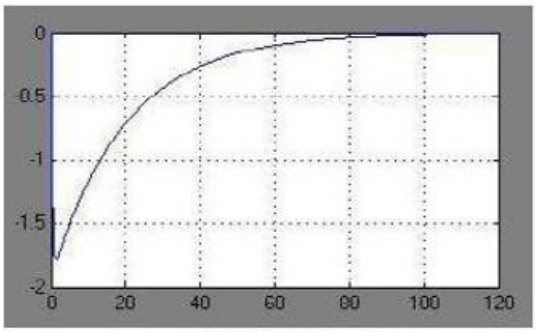
PROBLEM: The load dynamics of a single area system are Pr=2000 MW; NOL=1000MW; H=5s;f=50Hz; R=4%; TG=0.08s;TT=0.3s; Assume linear characteristics .The area has governor but not frequency control. It is subjected to an increase of 20MW. Construct simulink diagram and hence i) determine steady state frequency. ii) If speed governor loop was open, what would be the frequency drop?

iii) Prove frequency is zero if secondary controller is included.

OUTPUT:

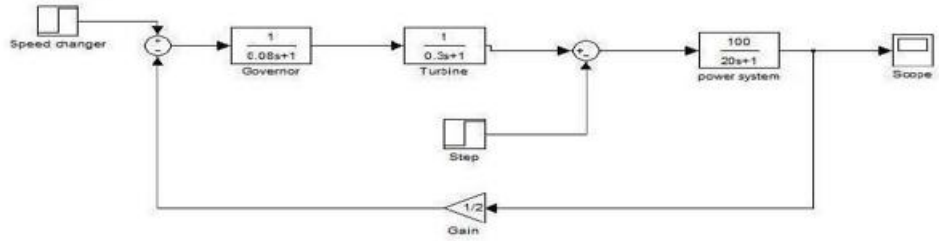
WITHOUT GAIN:

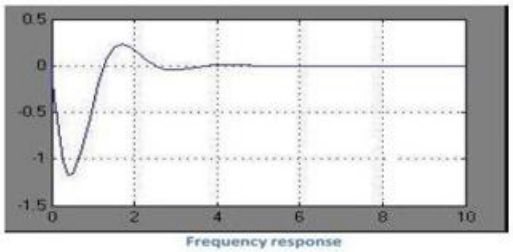




Frequency response

WITH GAIN





Frequency response

RESULT:

EX. NO.: 12 LOAD – FREQUENCY DYNAMICS OF TWO-AREA POWER SYSTEMS

AIM: To become familiar with the modeling and analysis of load frequency and tie line flow dynamics of a power systems with load frequency controller (LFC)

SOFTWARE REQUIRED: MATLAB

THEORY:

Primary control: The speed change from the synchronous speed initiates the governor control action resulting in all the participating generator-turbine units taking up the change in load, and stabilizes the system frequency.

Secondary control: It adjusts the load reference set points of selected turbine-generator units so as to give nominal value of frequency. The frequency control is a matter of speed control of the machines in the generating stations. The frequency of a power system is dependent entirely upon the speed in which the generators are rotated by their prime movers. All prime movers, whether they are steam or hydraulic turbines, are equipped with speed governors which are purely mechanical speed sensitive devices, to adjust the gate or control valve opening for the constant speed.

PROCEDURE:

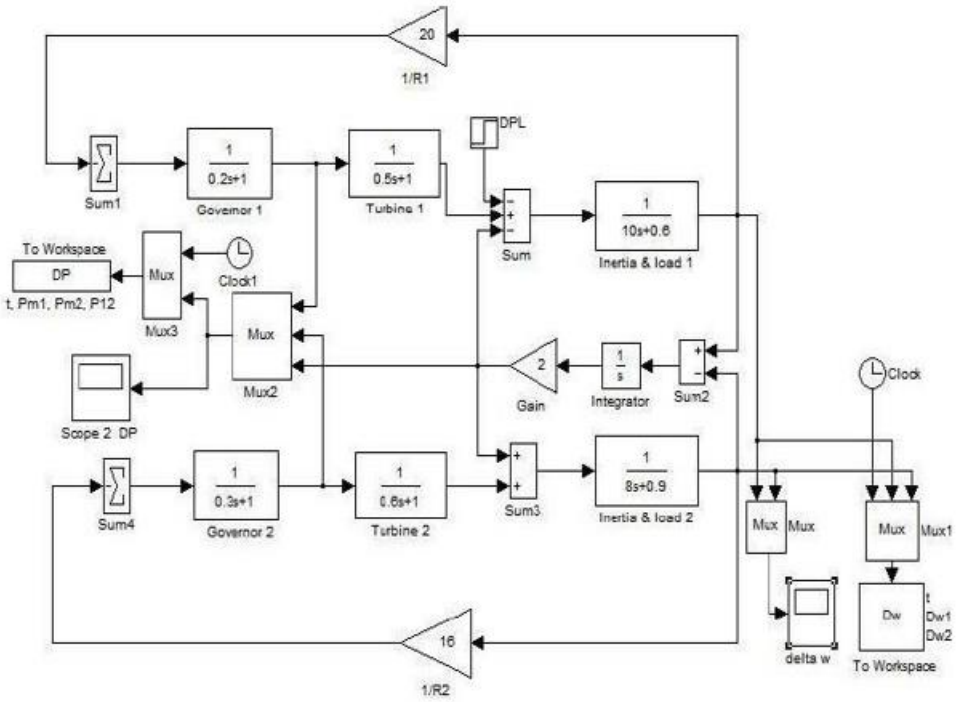
1. Enter the command window of the MATLAB.
2. Create a new Model by selecting File - New – Model.
3. Pick up the blocks from the simulink library browser and form a block diagram.
4. After forming the block diagram, save the block diagram.
5. Double click the scope and view the result.

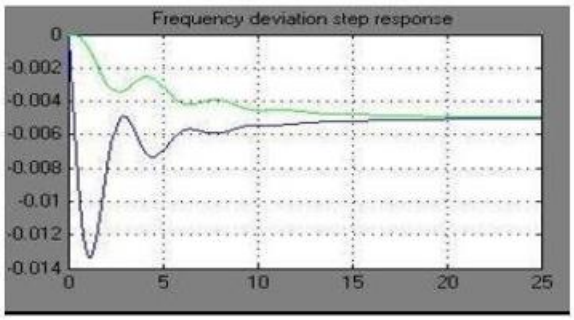
PROBLEM

1. A two area system connected by a tie line has the following parameters on a 1000 MVA base. R1=0.05pu, R2=0.0625pu,D1=0.6, D2=0.9,H1=5,H2=4; Base power1=Base power2=1000MVA, TG1=0.2s, TG2=0.3s, TT1=0.5s,TT2=0.6s. The units are operating in parallel at the nominal frequency of 50Hz. The synchronizing power coefficient is 2pu. A load change of 200MW occurs in area1. Find the

new steady state frequency and change in the tie line flow. Construct simulink block diagram and find deviation in frequency response for the condition mentioned.

OUTPUT:





RESULT:

EX. NO.: 13 UNSYMMETRICAL FAULT ANALYSIS

AIM: To become familiar with modeling and anal ysis of power systems under faulted condition and to compare the fault level, post-fault voltages and currents for unsymmetric fault

SOFTWARE REQUIRED: MATLAB

THEORY: The symmetrical fault can be analyzed on per phase basis using reactance diagram or by using per unit reactance diagram. The unsymmetrical faults are analyzed using symmetrical components. In symmetrical fault analysis, the reactance diagram of the power system is formed using the information in single line diagram.

PROCEDURE:

1. Enter the command window of the MATLAB.
2. Create a new M – file by selecting File - New – M – File
3. Type and save the program in the editor window.
4. Execute the program by either pressing Tools – Run.
5. View the results.

PROGRAM:

```
clc; clear all;
P=input('\n1.Symmetrical Fault\n2.UnSymmetrical Fault\n') switch(P)
case 1
disp('Symmetrical Fault')
Xg=input('Enter the reactance for generator = ');
Xt=input('Enter the reactance for transformer = ');
Xm=input('Enter the reactance for motor = ');
 I1=input('Enter the current value = ');
v=input('Enter the terminal vge of generator = ');
p=input('Enter the power factor value = '); I=I1*(cos(p)+1i*sin(p))
```

```
G=(Xg*I)+v
M=v-(Xt*I)+(Xm*I)
Ig=G/(Xg+Xt) Im=M/Xm
If=Ig+Im
case 2
F=input('\n1.Single Line- Ground Fault\n2Line - Line Fault\n3.Double Line - Ground
Fault\n')
switch(F)
case 1
disp('UnSymmetrical Fault') disp('Single Line- Ground Fault')
KVA=20*1000;
Kv=13.8; Ea=1;
a=-0.5+0.8666j;
a2=-0.5-0.8666j;
Z0=input('Enter the value of Z0 = ');
Z1=input('Enter the value of Z1 = ');
Z2=input('Enter the value of Z2 = ');
Bc=KVA/(1.732*Kv) Ia1=1/(Z0+Z1+Z2)
Ia0=Ia1; Ia2=Ia0;
If=3*(abs(Ia1)) I=If*Bc
Va1=Ea-(Ia1*Z1); Va2=-(Ia2*Z2); Va0=-(Ia0*Z0); Va=Va0+Va1+Va2;
Vb=Va0+(a2*Va1)+(a*Va2);
Vc=Va0+(a*Va1)+(a2*Va2);
Vab=Va-Vb;
Vbc=Vb-Vc;
 Vca=Vc-Va;
PV=(Kv/1.732)
Vab1=(abs(Vab))*PV
 Vbc1=(abs(Vbc))*PV
 Vca1=(abs(Vca))*PV case 2
disp('Line - Line Fault') KVA=20*1000; Kv=13.8;
Ea=1;
```

```
a=-0.5+0.8666j; a2=-0.5-0.8666j;
Z0=input('Enter the value of Z0 = ');
Z1=input('Enter the value of Z1 = ');
Z2=input('Enter the value of Z2 = ');
Bc=KVA/(3^(1/2)*Kv)
Ia1=0; Ia2=Ea/(Z1+Z2); Ia3=-Ia2;
Ia=0
Ib=Ia1+(a2*Ia2)+(a*Ia3) Ic=-Ib
If=Ib*Bc
Va1=Ea-(Ia2*Z1); Va2=-(Ia3*Z2); Va2=Va1;
Va0=0; Va=Va0+Va1+Va2;
Vb=(Va0+(a2*Va1)+(a*Va2)); Vc=Vb;
Vab=Va-Vb;
Vbc=Vb-Vc; Vca=Vc-Va; PV=(Kv/1.732);
Vab1=(abs(Vab))*PV
Vbc1=(abs(Vbc))*PV
 Vca1=(abs(Vca))*PV
case 3
disp('Double Line- Ground Fault')
 KVA=20*1000;
Kv=13.8; Ea=1;
a=-0.5+0.8666j; a2=-0.5-0.8666j;
Z0=input('Enter the value of Z0 = ');
Z1=input('Enter the value of Z1 = ');
Z2=input('Enter the value of Z2 = '); Bc=KVA/(3^(1/2)*Kv) g=(Z2*Z0)/(Z2+Z0);
h=Z1+g;
Ia1=Ea/h;
Va1=Ea-(Ia1*Z1);
Va2=Va1;
Va0=Va2;
b2=((-Ea)/Z2)+(Ia1*Z1);
b1=((-Ea)/Z0)+(Ia1*Z1);
```

```
Ia2=((b2)/(Z2));
Ia0=((b1)/(Z0)); Ia=Ia0+Ia1+Ia2
Ib=Ia0+(a2*Ia1)+(a*Ia2) Ic=Ia0+(a*Ia1)+(a2*Ia2)
If=Ib+Ic; IF=If*Bc
Va=Va0+Va1+Va2 Vb=0;
Vc=0;
Vab=Va-Vb; Vbc=Vb-Vc; Vca=Vc-Va; PV=(Kv/1.732)
Vab1=(abs(Vab))*PV Vbc1=(abs(Vbc))*PV Vca1=(abs(Vca))*PV end
end
```

OUTPUT:

RESULT:

EX. NO.: 14 STATE ESTIMATION: WEIGHTED LEAST SQUARE ESTIMATION

AIM: To obtain the bests possible estimate of state of the power system for the given set of measurement by weighted least squares method.

SOFTWARE REQUIRED: MATLAB

THEORY: State estimation is defined as the data processing algorithm for converting redundant meter reading and other available information into as estimate of the state of electrical power system. Real time measurement are collected in power system through SCADA system. Typical data includes real and reactive line flows and real and reactive bus injections and bus voltage magnitude. This telemetered data may contain errors. Theseerrors render the outputuseless. It is for this reason that, power system state estimation techniques have been developed. A commonly used criterion is that of minimizing the sum of the squares of the differences between estimated measurement quantities and actual measurement. This is known as "weighted least squares" criterion. The mathematical model of state estimation is based on the relation between the measurement variable and the state variable.

PROCEDURE:

1. Enter the command window of the MATLAB.
2. Create a new M – file by selecting File - New – M – File
3. Type and save the program in the editor window.
4. Execute the program by either pressing Tools – Run.
5. View the results.

PROGRAM:

```
%% Weighted Least Square (WLS) State Estimation
clc; clear;
% 1. Power System Data (Admittance Matrix - Ybus)
% Example 3-bus system (Simplified impedances)
% Line 1-2: z=0.01+j0.03, 1-3: z=0.02+j0.05, 2-3: z=0.01+j0.03
Ybus = [ 6.25-18.75i, -3.12+9.37i, -3.12+9.37i;
-3.12+9.37i, 6.25-18.75i, -3.12+9.37i;
```

```
-3.12+9.37i, -3.12+9.37i, 6.25-18.75i];
G = real(Ybus);
 B = imag(Ybus);
% 2. Measurement Data (z) and Variance (R) %
Measurements: [V1; V2; P12; P13; Q12; Q13]
z = [1.0; 1.0; 0.5; 0.4; 0.1; 0.05];
sigma = [0.001; 0.001; 0.01; 0.01; 0.01; 0.01]; % Standard deviations
R = diag(sigma.^2);
W = inv(R); % Weight matrix
 % 3. Initialize States [Angles(2:n); Magnitudes(1:n)]
% Bus 1 is slack (theta1 = 0)
x = [0; 0; 1.0; 1.0; 1.0]; % [th2, th3, V1, V2, V3]
 n_states = length(x); n_meas = length(z); iter = 0; max_iter = 10; tol = 1e-6;
 converged = false;
fprintf('Iter | Objective J(x)\n----------\n');
 % 4. Gauss-Newton Iteration
while ~converged && iter < max_iter iter = iter + 1; th = [0; x(1); x(2)];
% Angles V = [x(3); x(4); x(5)];
% Voltages
% h(x) - Calculation of estimated measurements
 h = zeros(n_meas, 1);
h(1) = V(1); % V1
h(2) = V(2); % V2
% P12 = V1*V2*(G12*cos(th1-th2) + B12*sin(th1-th2)) - G12*V1^2 (Simplified)
 h(3) = V(1)*V(2)*(G(1,2)*cos(th(1)-th(2)) + B(1,2)*sin(th(1)-th(2)));
h(4) = V(1)*V(3)*(G(1,3)*cos(th(1)-th(3)) + B(1,3)*sin(th(1)-th(3)));
h(5) = V(1)*V(2)*(G(1,2)*sin(th(1)-th(2)) - B(1,2)*cos(th(1)-th(2)));
h(6) = V(1)*V(3)*(G(1,3)*sin(th(1)-th(3)) - B(1,3)*cos(th(1)-th(3)));
% Residuals
 r = z - h;
% 5. Jacobian Matrix  H = dh/dx (Numerical approximation or Analytical)
% For brevity, a simplified Jacobian placeholder is used:
```

```
% In a real project, calculate partial derivatives of h w.r.t th and V
 H = zeros(n_meas, n_states);
% Example partial derivatives (simplified for V1, V2)
H(1,3) = 1; % dV1/dV1
H(2,4) = 1; % dV2/dV2
 % ... (Populate other H elements with partial derivatives) ...
 % Note: Use analytical derivatives for P and Q flows for accuracy.
% Dummy H for demonstration (Replace with actual derivatives)
H = rand(n_meas, n_states);
% 6. Gain Matrix and Update
 Gain = H' * W * H;
 delta_x = inv(Gain) * H' * W * r;
x = x + delta_x;
J = r' * W * r;
 fprintf('%4d | %10.6f\n', iter, J);
 if max(abs(delta_x)) < tol converged = true;
 end
end
 % 7. Display Results
disp('Estimated States (Angles in rad, Voltages in p.u.):');
disp(x);
```

OUTPUT:

RESULT:

EX. NO.: 15 ELECTROMAGNETIC TRANSIENTS IN POWER SYSTEMS: TRANSMISSION LINE ENERGIZATION

AIM: To study and understand the electromagnetic transient phenomena in power systems caused due to switching and fault by PSCAD software.

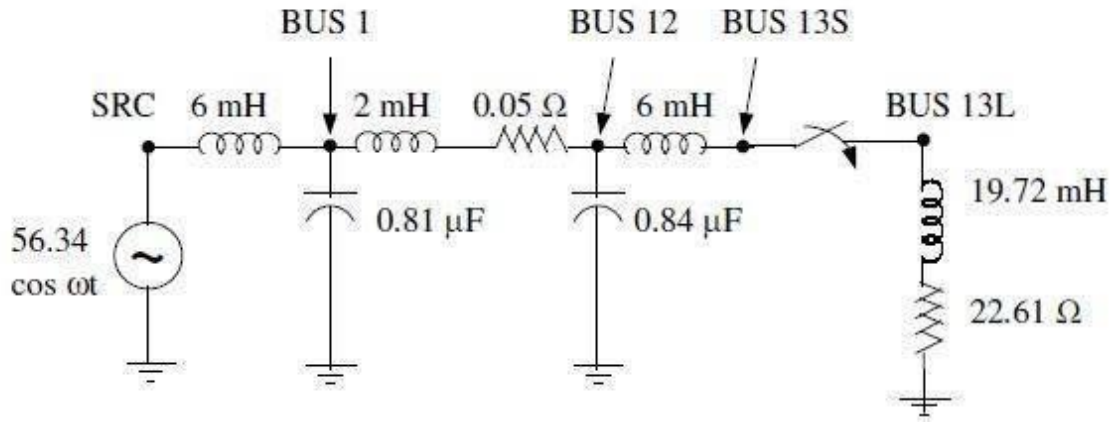
SOFTWARE REQUIRED: MATLAB

THEORY: Intentional and inadvertent switching operations in EHV systems initiate over voltages, which might attain dangerous values resulting in destruction of apparatus. Accurate computation of these voltages is essential for proper sizing, co-ordination of insulation of various equipments and specification of protective devices. Meaningful use of EHV is dependent on modeling philosophy built into a computer program. The models of equipments must be detailed enough to reproduce actual conditions successfully – an important aspect where a general purpose digital computer program scores over transient network analyzers. The program employs a direct integration time-domain technique evolved by Dommel. The essence of this method is discretization of differential equations associated with network elements using trapezoidal rule of integration and solution of the resulting equations for the unknown voltages. Any network which consists of interconnections of resistances, inductances, capacitances, single and multiphase pi circuits, distributed parameter lines, and certain other elements can be solved. To keep the explanations simple, however, single phase network elements will be used, rather than the complex multiphase network elements.

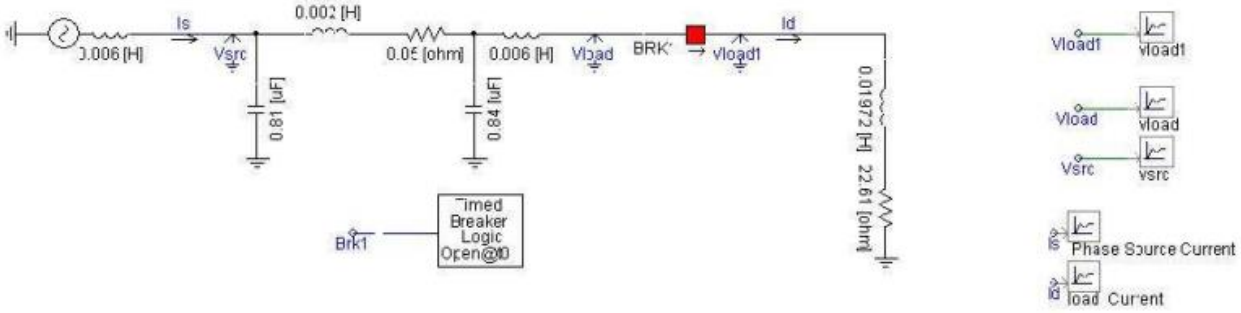
Problem:

Prepare the data for the network given in the given figure. Obtain the plots of source voltage, load bus voltage and load current following the energization of a single-phase load. Comment on the results. Double the source inductance and obtain the plots of the variables mentioned earlier. Comment on the effect of doubling the source inductance. Energization of a single phase 0.95 pf load from a non ideal source and a more realistic line representation (lumped R, L, C) using PSCAD software.

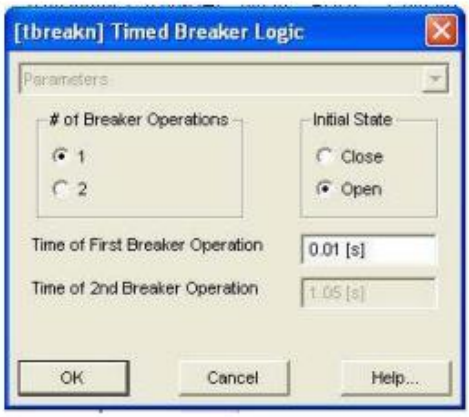
(i) Circuit Diagram

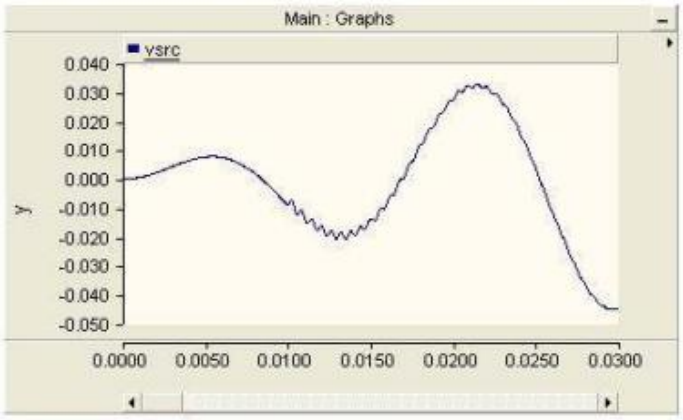


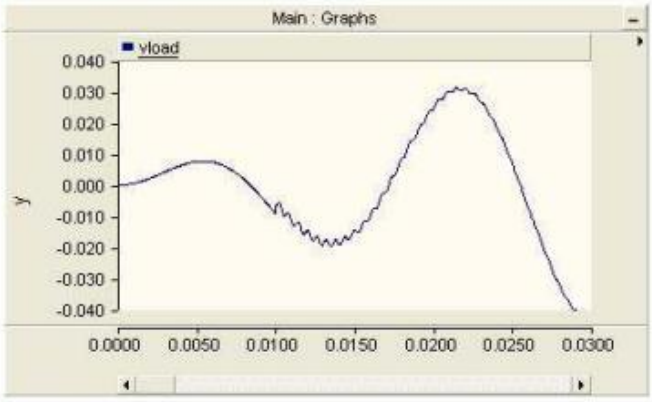
PSCAD MODEL

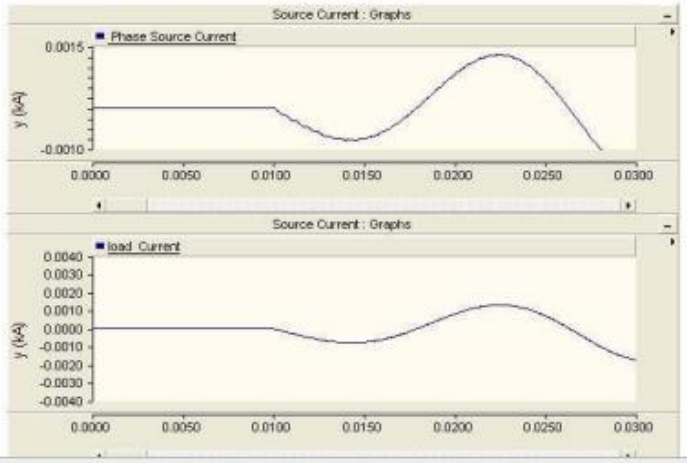


OUTPUT:


[tbreakn] Timed Breaker Logic
Parameters
# of Breaker Operations
1
2
Initial State
Close
Open
Time of First Breaker Operation
0.01 [s]
Time of 2nd Breaker Operation
1.05 [s]
OK
Cancel
Help...


Main : Graphs
vsrc
y
0.040
0.030
0.020
0.010
0.000
-0.010
-0.020
-0.030
-0.040
-0.050
0.0000
0.0050
0.0100
0.0150
0.0200
0.0250
0.0300


Main : Graphs
vload
y
0.040
0.030
0.020
0.010
0.000
-0.010
-0.020
-0.030
-0.040
0.0000
0.0050
0.0100
0.0150
0.0200
0.0250
0.0300


Source Current : Graphs
Phase Source Current
y (kA)
0.0015
-0.0010
0.0000
0.0050
0.0100
0.0150
0.0200
0.0250
0.0300
Source Current : Graphs
load Current
y (kA)
0.0040
0.0030
0.0020
0.0010
0.0000
-0.0010
-0.0020
-0.0030
-0.0040

RESULT: